

Rapport de Presentation d'OpenStack

Projet réalisé par :

THEUBO Ghislain	Superviseur: Caleb Fonyuy
-----------------	----------------------------------

Table des matières

I. Chapitre 1:	3
II. Chapitre2 :	3
III. Jeux de données étudiés	3
IV. Description détaillée du jeu de données choisi	4
V. Architecture globale de l'application	6
VI. Description des traitements mis en place	7
1. Prétraitement du jeu de données	7
2. Entraînement du modèle	9
3. Test du modèle	12
VII. Description du WebChat	13
1. Structure du repository :	13
2. Architecture du chat :	14
3. Comment lancer l'application :	14
4. Description du fonctionnement Chat	15
VIII. Les limites de notre solution :	18
IX. Les impacts environnementaux et sociétaux :	18
1. Impact social de l'application	18
2. Impact environnemental immédiat de l'application	19
3. Impact environnemental à long terme	20
Sources	21

PROCESSUS D'INSTALLATION SUIVI JUSQU'ICI

D'après la sortie de la commande `ifconfig`, l'interface physique sur ton ordinateur semble être `enp0s31f6` (une interface Ethernet), ou `wlp2s0` (une interface sans fil, Wi-Fi). L'interface `enp0s31f6` est généralement utilisée pour la connexion Ethernet filaire, tandis que `wlp2s0` est utilisée pour la connexion Wi-Fi.

Pour un réseau bridgé dans une configuration KVM, tu voudras probablement utiliser l'interface `enp0s31f6` si tu es connecté à Internet via une connexion filaire. Si tu préfères utiliser la connexion sans fil `wlp2s0`, tu peux également l'utiliser, mais il faut noter que la configuration de réseaux bridgés avec Wi-Fi peut parfois poser des problèmes en fonction des pilotes et des paramètres du système.

Pour configurer un réseau bridgé (bridged network) sur `enp0s31f6` :

Créer un pont réseau (bridge) en utilisant `brctl` :

```
sudo apt install bridge-utils
```

```
sudo brctl addbr br0
```

```
sudo brctl addif br0 enp0s31f6
```

- 1.
2. **Configurer l'interface réseau `br0` :**

Ouvre ou crée un fichier de configuration pour l'interface `br0` dans `/etc/network/interfaces` (sur Ubuntu) :

```
sudo nano /etc/network/interfaces
```

○

Ajoute les lignes suivantes :

```
auto br0
```

```
iface br0 inet dhcp
```

```
bridge_ports enp0s31f6
```

○

3. Cela configure un pont réseau **br0** qui utilise **enp0s31f6** comme interface physique et qui obtient une adresse IP via DHCP.

Redémarrer le réseau :

Pour appliquer les changements, redémarre le service réseau :

```
sudo systemctl restart networking
```

4.

Vérification

Une fois ces étapes effectuées, tu devrais voir **br0** comme nouvelle interface réseau dans **ifconfig**. Tu peux vérifier que la connectivité fonctionne correctement entre ton hôte et tes VM en pingant une autre machine (par exemple, le routeur ou une autre VM sur le même réseau).

Étape suivante

Ensuite, dans **virt-manager**, lorsque tu crées tes machines virtuelles, il te suffit de sélectionner l'interface **br0** pour qu'elles utilisent ce pont réseau, et ainsi qu'elles puissent communiquer entre elles et avec l'extérieur.

Si tu préfères utiliser Wi-Fi avec **wlp2s0**, tu peux suivre une méthode similaire, mais cela pourrait être plus complexe selon la configuration de ton Wi-Fi et des restrictions de ton matériel.

Il semble que ton pont **br0** est bien configuré avec l'interface **enp0s31f6** et qu'il est prêt à être utilisé. Voici quelques étapes pour t'assurer que tout est configuré correctement et pour préparer tes nœuds pour OpenStack.

1. Vérifier que le pont a une adresse IP

Vérifie que **br0** a bien une adresse IP assignée (si tu utilises DHCP). Exécute cette commande pour afficher les interfaces réseau :

```
ip a
```

Tu devrais voir **br0** avec une adresse IP associée. Si **br0** ne possède pas d'adresse IP, tu peux l'ajouter avec la commande suivante :

```
sudo dhclient br0
```

2. Configurer les machines virtuelles sur le pont **br0**

Maintenant que **br0** est configuré, tu peux connecter tes machines virtuelles à ce pont pour qu'elles puissent communiquer avec ton réseau local.

- **Avec Virt-Manager :**
 - Ouvre **Virt-Manager** et sélectionne ou crée une machine virtuelle.
 - Lorsque tu configures le matériel de la machine virtuelle (en particulier la carte réseau), choisis **"Bridge"** et sélectionne **br0** comme pont. Cela permettra à la machine virtuelle de se connecter au même réseau que l'interface physique.
- **Avec **virsh** (ligne de commande) :**

Crée une interface réseau pour la machine virtuelle en spécifiant le pont **br0**.

Exemple de configuration pour une machine virtuelle déjà créée :

```
virsh attach-interface <vm_name> bridge br0 --model virtio --mac <mac_address>
```

-
- Remplace **<vm_name>** par le nom de ta machine virtuelle et **<mac_address>** par l'adresse MAC de la carte réseau de la VM si nécessaire.

3. Vérifier la connectivité des VMs

Après avoir configuré les interfaces réseau pour tes machines virtuelles, tu peux vérifier la connectivité en les démarrant et en testant avec un **ping** vers une autre machine ou une adresse IP externe (par exemple **8.8.8.8** pour Google DNS).

4. Configurer OpenStack sur tes nœuds

Une fois les nœuds configurés avec KVM/QEMU, tu peux préparer chaque machine virtuelle comme un nœud dans un déploiement OpenStack. L'étape suivante

consistera à installer les composants nécessaires d'OpenStack sur chaque nœud (contrôleur, calcul, stockage).

Est-ce que tu souhaites que je t'aide à configurer OpenStack sur ces nœuds ou à passer à une autre étape du processus ?

Pour vérifier l'adresse MAC d'une machine virtuelle, tu as plusieurs méthodes :

1. Avec virt-manager (interface graphique)

1. Ouvre **virt-manager**.
2. Sélectionne la machine virtuelle dont tu veux connaître l'adresse MAC.
3. Clique sur le bouton pour afficher les **Détails** ou **Configuration** de la machine virtuelle.
4. Dans la liste du matériel (hardware), clique sur l'interface réseau (NIC).
5. L'adresse MAC est généralement affichée dans les propriétés de cette interface.

2. Avec virsh (ligne de commande)

Utilise la commande suivante en remplaçant **<nom_VM>** par le nom de ta machine virtuelle :

```
virsh domiflist <nom_VM>
```

Cette commande affiche une table listant les interfaces réseau de la machine virtuelle, avec les colonnes indiquant le type d'interface, le réseau utilisé et l'adresse MAC.

Par exemple :

```
$ virsh domiflist ma_vm
```

Interface	Type	Source	Model	MAC

vnet0	bridge	br0	virtio	52:54:00:ab:cd:ef

L'adresse MAC se trouve dans la colonne **MAC**.

Utilise la méthode qui te convient le mieux pour obtenir l'adresse MAC de la VM.

Oui, il te faudra attacher une interface pour chaque machine virtuelle si tu souhaites qu'elles se connectent toutes via le même pont réseau (**br0**). Voici quelques points à considérer :

Attachement pour chaque VM :

Si une VM n'a pas encore d'interface connectée au pont **br0**, alors tu devras exécuter une commande similaire à celle-ci pour chaque machine, en veillant à utiliser une adresse MAC unique pour chaque interface. Par exemple :

```
virsh attach-interface <nom_VM> bridge br0 --model virtio --mac <MAC_unique>
```

1. Assure-toi de remplacer **<nom_VM>** par le nom de la VM et **<MAC_unique>** par une adresse MAC différente pour chaque VM.

Vérification :

Tu peux vérifier quelles interfaces sont attachées à une VM en utilisant la commande suivante :

```
virsh domiflist <nom_VM>
```

2. Cela te permettra de confirmer que l'interface connectée au pont **br0** a bien été ajoutée.

Changements persistants :

Par défaut, l'attachement via **virsh attach-interface** affecte l'état en direct (live) de la VM. Si tu souhaites que la modification soit persistante (c'est-à-dire qu'elle soit conservée après un redémarrage de la VM), ajoute l'option **--config** :

```
virsh attach-interface <nom_VM> bridge br0 --model virtio --mac <MAC_unique> --config
```

- 3.

En résumé, pour que toutes tes machines virtuelles se connectent au pont **br0**, répète l'opération pour chacune d'elles en veillant à ce que chaque interface ait une adresse MAC unique.

Voici un récapitulatif détaillé des étapes à suivre maintenant que tes machines virtuelles (nœuds) sont créées, configurées en réseau via le pont (br0) et que le système d'exploitation est installé sur chacune d'elles.

Étape 3 : Préparer chaque nœud pour OpenStack

1. Mettre à jour le système

Sur **chaque VM**, connecte-toi et exécute :

```
sudo apt update && sudo apt upgrade -y
```

Cela garantit que tous les paquets sont à jour.

2. Vérifier et, si besoin, configurer les interfaces réseau

- **Vérifie la connectivité interne :**
Depuis chaque nœud, teste la communication avec d'autres VMs ou avec une adresse externe (par exemple, `ping 8.8.8.8`) pour t'assurer que le réseau fonctionne correctement.
- **Configuration locale (si nécessaire) :**
Dans la plupart des cas, l'interface réseau de la VM (souvent nommée `eth0` ou `ensX`) sera déjà configurée via DHCP.
Si tu souhaites simuler un réseau privé dédié à OpenStack, tu peux configurer une adresse IP statique ou même créer un petit pont interne sur la VM (mais ce n'est généralement pas nécessaire puisque le pont côté hôte, ici `br0`, gère la connectivité entre les VMs).

3. Installer les dépendances nécessaires

Sur chaque nœud, installe les outils essentiels pour interagir avec OpenStack :

```
sudo apt install python3-openstackclient python3-pip ntp
```

```
sudo pip3 install -U pip
```

Ces paquets te permettront d'utiliser la CLI OpenStack et d'assurer la synchronisation horaire.

4. Configurer la synchronisation de l'horloge

La synchronisation des horloges est cruciale pour OpenStack. Sur chaque nœud, assure-toi que NTP est activé :


```
sudo systemctl enable ntp
```

```
sudo systemctl start ntp
```

Vérifie ensuite que l'heure est correctement synchronisée (par exemple, avec la commande `date`).

Étape 4 : Installer les services OpenStack sur le nœud de contrôle

Choisis l'une de tes VMs pour être le **nœud de contrôle**. Sur ce nœud :

1. Installer Git

Si ce n'est pas déjà fait :

```
sudo apt install git
```

2. Cloner DevStack

DevStack est un moyen rapide de déployer OpenStack pour tester ou en développement.

```
git clone https://opendev.org/openstack/devstack
```

```
cd devstack
```

3. Créer et configurer le fichier `local.conf`

Crée un fichier nommé `local.conf` dans le dossier `devstack` :

```
nano local.conf
```

Ajoute-y le contenu suivant (en adaptant les mots de passe si nécessaire) :

```
[[local|localrc]]
```

ADMIN_PASSWORD=admin

DATABASE_PASSWORD=admin

RABBIT_PASSWORD=admin

SERVICE_PASSWORD=admin

4. Lancer l'installation

Toujours depuis le répertoire `devstack`, exécute :

```
./stack.sh
```

Le script va télécharger et configurer tous les services OpenStack sur le nœud de contrôle. Cette étape peut prendre un certain temps.

Étape 5 : Configurer les nœuds de calcul

Pour chaque VM destinée à être un **nœud de calcul** (Compute Node) :

1. Installer le service Nova Compute

Exécute sur chaque nœud de calcul :

```
sudo apt install nova-compute
```

2. Configurer le fichier `/etc/nova/nova.conf`

Ouvre le fichier de configuration avec, par exemple :

```
sudo nano /etc/nova/nova.conf
```

Ajoute ou modifie les sections suivantes (remplace `<IP_du_noeud_de_calcul>` par l'adresse IP réelle de ce nœud et ajuste le `transport_url` si nécessaire – l'exemple suppose que le nœud de contrôle est accessible par le nom `controller`) :

[DEFAULT]

```
transport_url = rabbit://openstack:admin@controller
```

```
my_ip = <IP_du_noeud_de_calcul>
```

```
vnc_enabled = True
```

```
vncserver_listen = 0.0.0.0
```

```
vncserver_proxyclient_address = $my_ip
```

3. Redémarrer le service Nova Compute

Applique les modifications en redémarrant le service :

```
sudo systemctl restart nova-compute
```

Étape 6 : Vérifier l'installation d'OpenStack

Sur le **nœud de contrôle**, utilise la commande suivante pour voir si les nœuds de calcul sont correctement enregistrés et fonctionnels :

```
openstack compute service list
```

Tu devrais voir une liste des services de calcul avec leur statut. Cela te confirmera que la communication entre le nœud de contrôle et les nœuds de calcul est effective.

En résumé

1. Sur chaque nœud (VM) :

- Mets à jour le système.
- Vérifie que la connectivité réseau fonctionne.
- Installe les dépendances (python3-openstackclient, python3-pip, ntp).
- Configure NTP pour la synchronisation de l'horloge.

2. Sur le nœud de contrôle :

- Installe Git et clone DevStack.
- Crée le fichier `local.conf` avec tes mots de passe.
- Exécute `./stack.sh` pour installer OpenStack.

3. Sur les nœuds de calcul :

- Installe `nova-compute`.
- Configure `/etc/nova/nova.conf` pour pointer vers le nœud de contrôle.
- Redémarre le service `nova-compute`.

4. Vérifie l'installation sur le nœud de contrôle avec `openstack compute service list`.

Une fois ces étapes réalisées, ton environnement OpenStack de test devrait être opérationnel ! Si tu as besoin d'aide supplémentaire pour une étape en particulier ou si tu rencontres un problème, n'hésite pas à demander.

SEMAINE 2 EVOLUTION OPEN STACK (REUNION LE 20/02/2025)

Pour comprendre comment **migrer une machine virtuelle (VM)** d'un nœud (Compute Node) OpenStack à un autre, il faut d'abord saisir l'architecture d'OpenStack et le rôle joué par l'hyperviseur (KVM, QEMU, etc.). Voici les points clés :

1. Rappels sur l'architecture OpenStack

1. Nœuds de calcul (Compute Nodes)

- Hébergent les VM.
- Exécutent Nova Compute (service Nova) qui orchestre la création, la gestion et la suppression des instances.
- S'appuient sur un hyperviseur (souvent KVM/QEMU) pour la virtualisation.

2. Nœud de contrôle (Controller Node)

- Héberge les services centraux : base de données, API Nova, API Neutron, service d'authentification Keystone, etc.

- C'est lui qui reçoit les requêtes (par exemple, « lance une VM », « migre cette VM »), puis coordonne avec les nœuds de calcul.

3. Stockage

- Selon la configuration, on peut avoir un stockage **ephemeral** (disque local du Compute Node) ou un stockage **partagé** (NFS, Ceph, etc.) pour les disques des VM.
 - Les volumes Cinder peuvent aussi être attachés aux VM et résider sur un backend de stockage (LVM, Ceph, iSCSI, etc.).
-

2. Les différents types de migration

2.1. Cold Migration

- **Cold migration** : la VM est arrêtée, puis déplacée (disque, configuration, etc.) vers un autre nœud de calcul, puis redémarrée.
- Avantage : plus simple à mettre en place (pas besoin de partage de stockage sophistiqué).
- Inconvénient : la VM est indisponible pendant la migration.

2.2. Live Migration

- **Live migration** : la VM continue de tourner pendant la quasi-totalité du transfert de sa RAM d'un hôte source à un hôte cible, puis subit une toute petite interruption (quelques millisecondes à quelques secondes) lors du basculement final.
 - Avantage : temps d'indisponibilité minimal.
 - Inconvénient : nécessite souvent une configuration réseau et un stockage partagés plus complexes (ou au moins un mécanisme de copie de disque « à chaud »).
-

3. Rôle de l'hyperviseur dans la migration

1. Gestion de la mémoire :

- Avec KVM/QEMU, la mémoire de la VM est copiée par le processus QEMU d'un nœud vers le QEMU du nœud cible.
- Tant que la synchronisation n'est pas terminée, la VM « source » continue de fonctionner et les pages modifiées sont re-copiées jusqu'à ce que la différence soit négligeable.

2. Gestion du disque :

- Si un stockage **partagé** est utilisé (NFS, Ceph, GlusterFS, etc.), la VM utilise le même disque virtuel depuis la nouvelle machine : pas besoin de le recopier intégralement.
- Si le disque est **local** (ephemeral) et qu'on fait du live migration sans stockage partagé, Nova peut tenter de copier le disque à travers le réseau, ce qui est plus long et plus complexe (certaines configurations ne le permettent pas ou le font en « block live migration »).

3. Gestion du réseau :

- L'hyperviseur reconfigure l'interface virtuelle de la VM sur le nouveau nœud pour conserver la même adresse MAC et la même configuration IP.
- Neutron (le service réseau) met à jour les flux Open vSwitch/OVS ou Linux Bridge pour router le trafic de la VM vers le nouveau nœud.

4. Coordination via Nova :

- Nova gère la requête de migration (cold ou live) et appelle les hooks nécessaires côté hyperviseur (QEMU/KVM).
- Nova met à jour sa base de données interne pour signaler que la VM est désormais sur le nouveau nœud.

4. Conditions requises pour la Live Migration

1. Compatibilité matérielle

- Les CPU des nœuds source et cible doivent être suffisamment proches (ex. même famille Intel ou AMD) pour que l'instruction set ne change pas.
- Sinon, on utilise des modes de CPU virtuels (CPU modes) pour masquer certaines instructions avancées.

2. Stockage partagé ou mécanisme de copie

- Avec un stockage partagé (ex. Ceph RBD, NFS, etc.), le disque de la VM est déjà accessible depuis tous les nœuds. Seule la RAM doit être transférée.
- Sans stockage partagé, on doit configurer la « block live migration » : la RAM et le disque sont copiés en direct vers l'hôte cible.

3. Réseau

- Le nouveau nœud de calcul doit pouvoir rejoindre le même réseau virtuel.
- La VM doit conserver la même adresse IP, la même configuration de sécurité (Security Groups), etc.
- Neutron gère l'association du port virtuel de la VM au nouveau nœud.

4. Configuration de Nova

- Il faut autoriser la migration dans `/etc/nova/nova.conf` (live_migration_flag, block_migration_flag, etc.).
- Les services `nova-scheduler` et `nova-conductor` doivent être au courant des règles d'assignation des hôtes.

5. Scénario de migration dans un même réseau (Grid5000 ou autre)

1. Déploiement OpenStack :

- Supposons que vous avez un cluster sur Grid5000 : plusieurs nœuds physiques, chacun configuré en nœud de calcul.
- Le nœud de contrôle gère l'orchestration.

2. Création d'une VM sur un nœud X :

- Via la commande `openstack server create ...` ou via Horizon (le dashboard).
- La VM se lance et utilise soit un disque ephemeral local, soit un volume sur un backend de stockage partagé.

3. Migration vers un nœud Y :

Commande de migration (ex. live) :

```
openstack server migrate --live <nom_ou_id_VM> <hôte_cible> --shared-migration
```

ou

```
nova live-migration <nom_ou_id_VM> <hôte_cible>
```

- (selon la version de Nova et du client OpenStack).
- Nova va orchestrer la copie de la RAM (et éventuellement du disque) via l'hyperviseur.
- Le réseau virtuel est reconfiguré pour que la VM conserve son IP.

4. Rôle du stockage :

- Si vous utilisez Ceph RBD, la VM aura un disque virtuel RBD accessible depuis tous les nœuds. La migration sera plus rapide.
- Si vous utilisez un disque local, Nova copiera le disque via le réseau (block migration).

5. Résultat :

- La VM est désormais sur l'hôte Y.
- L'adresse IP et la configuration ne changent pas (c'est transparent pour l'utilisateur).
- L'hyperviseur sur l'hôte X a libéré la RAM et le CPU.

6. Conclusion et logique de mise en place

- **Hyperviseur (KVM/QEMU)** : gère la virtualisation des VM et le transfert de la mémoire lors de la live migration.
- **Nova** : reçoit la requête de migration, valide les prérequis (compatibilité CPU, stockage, etc.), et orchestre le processus.
- **Neutron** : reconfigure la couche réseau pour que la VM conserve son IP et reste joignable.
- **Stockage** : idéalement partagé pour faciliter la migration (sinon, configuration de block migration).

Pour résumer :

1. Configurez Nova pour autoriser la migration (fichiers de config).
2. Assurez-vous d'avoir un stockage partagé ou la configuration block migration.
3. Vérifiez la compatibilité CPU entre les nœuds.
4. Utilisez la commande de migration (live ou cold) pour déplacer la VM.

Ainsi, dans un environnement comme Grid5000 (ou tout autre cluster), dès lors que vos nœuds de calcul partagent le même backend de stockage ou sont configurés pour la block migration, vous pouvez migrer les VMs d'un nœud à l'autre, soit en « live » (presque sans interruption) soit en « cold » (avec arrêt de la VM). L'hyperviseur KVM/QEMU est au cœur du processus technique de transfert de la mémoire et de l'état de la VM, tandis que Nova et Neutron orchestrent la logique globale et le réseau.

En se basant sur le contenu du fichier « OpenStack Essentials », voici comment comprendre et mettre en place la migration d'une machine virtuelle (VM) dans un environnement OpenStack, ainsi que le rôle de l'hyperviseur dans ce processus :

1. Vue d'ensemble de l'architecture (selon OpenStack Essentials)

- **Composants clés :**

Le livre détaille que OpenStack est divisé en plusieurs composants :

- Le **contrôleur** qui centralise l'authentification (Keystone), la gestion des images (Glance), la planification et le suivi des instances (Nova), etc.
- Les **nœuds de calcul** (Compute Nodes) qui hébergent les VM via un hyperviseur (généralement KVM/QEMU).
- Le service **Neutron** qui gère la connectivité réseau entre ces composants et les instances.

- **Stockage et réseaux :**

Le stockage peut être local (éphémère) ou partagé (via Ceph, NFS, etc.), et la connectivité réseau est assurée par Neutron qui peut utiliser des solutions comme Open vSwitch pour créer des réseaux isolés et assurer la communication entre les VM.

2. Migration de VM dans OpenStack

Selon « OpenStack Essentials », lorsqu'on parle de migration de VM, on distingue deux scénarios :

a) Cold Migration

- **Procédure :**

La VM est arrêtée, transférée (son image disque, sa configuration) vers un autre nœud, puis redémarrée sur ce dernier.

- **Avantage :**

Simplicité d'implémentation, notamment lorsque le stockage n'est pas partagé.

- **Inconvénient :**

La VM est indisponible durant l'opération.

b) Live Migration

- **Procédure :**
La migration est effectuée pendant que la VM est en cours d'exécution. La mémoire (et, dans certains cas, le stockage) est transférée progressivement vers le nœud cible, avec une interruption minimale à la fin du transfert.
 - **Exigences :**
 - **Stockage partagé :** Idéalement, les images de disque sont stockées sur un système accessible par tous les nœuds (par exemple, Ceph ou NFS), de sorte que seule la mémoire doit être migrée.
 - **Compatibilité matérielle :** Les nœuds doivent avoir des CPU compatibles pour éviter des problèmes d'instruction.
 - **Configuration réseau :** Le service Neutron doit pouvoir reconfigurer l'acheminement des flux réseau pour que la VM conserve la même adresse IP et le même point d'accès.
-

3. Rôle de l'hyperviseur dans la migration

D'après « OpenStack Essentials » :

- **Hyperviseur (KVM/QEMU) :**
 - **Transfert de l'état de la VM :** Lors d'une live migration, c'est l'hyperviseur qui prend en charge la copie progressive de la mémoire de la VM depuis le nœud source vers le nœud cible.
 - **Gestion des interfaces virtuelles :** Il s'assure que la configuration (adresse MAC, etc.) reste identique sur le nœud cible, garantissant ainsi la continuité de la connectivité réseau.
 - **Interaction avec Nova :** L'hyperviseur répond aux appels de Nova qui orchestre l'ensemble du processus de migration (live ou cold).
 - **Nova (Compute Service) :**
 - Coordonne la migration en vérifiant que toutes les conditions sont remplies (compatibilité, stockage accessible, etc.) et en déclenchant les mécanismes de transfert de l'état de la VM via l'hyperviseur.
 - Met à jour sa base de données pour refléter le changement de nœud.
-

4. Mise en place pratique dans un environnement multi-nœuds

Imaginons que vous déployez OpenStack sur Grid5000 avec plusieurs nœuds de calcul :

1. Création de la VM :

La VM est lancée sur un nœud de calcul via Nova, en utilisant une image fournie par Glance.

- Si le stockage est partagé (par exemple avec Ceph RBD), le disque virtuel est accessible de tous les nœuds.

2. Migration de la VM :

- **Live Migration :**

Vous pouvez utiliser la commande de migration live (par exemple via le client Nova ou OpenStack CLI) pour déplacer la VM d'un nœud à un autre. Nova demande alors à l'hyperviseur source de transférer progressivement la mémoire vers le nœud cible.

- **Cold Migration :**

La VM est arrêtée, transférée, puis redémarrée sur le nœud cible.

3. Rôle de Neutron :

Pendant ou après la migration, Neutron ajuste la configuration réseau (routage, ports virtuels, etc.) pour que la VM reste accessible avec la même adresse IP.

4. Transparence pour l'utilisateur final :

L'objectif est que la migration soit transparente : l'utilisateur final ne remarque aucune différence en termes de connectivité ou de performance (surtout en live migration).

5. Conclusion

En résumé, d'après « OpenStack Essentials » :

- **La migration de VM** peut être effectuée en mode cold ou live, en fonction des besoins et de la configuration de votre environnement (stockage partagé, compatibilité matérielle, etc.).
- **L'hyperviseur** (KVM/QEMU) joue un rôle central en transférant l'état (mémoire et, le cas échéant, le disque) de la VM d'un nœud à l'autre.
- **Nova** orchestre le processus de migration en validant les conditions préalables et en coordonnant l'action de l'hyperviseur.
- **Neutron** assure que la connectivité réseau reste intacte pendant et après la migration.

Pour mettre cela en place, assurez-vous que votre infrastructure respecte les prérequis pour la migration live (stockage partagé, réseaux configurés, CPU

compatibles, etc.), et utilisez les commandes et options fournies par Nova pour lancer et surveiller le processus.

Cette logique et ces mécanismes, détaillés dans le livre, garantissent une migration efficace et transparente des instances dans un environnement OpenStack multi-nœuds.

SEMAINE 3

COMMENT ACTIVER KVM SUR NOVA

Traduction et Explication

Traduction en français

À faire

Ce contenu est plutôt un guide d'installation et devrait probablement être déplacé.

KVM est configuré comme l'hyperviseur par défaut pour Compute.

Remarque

Ce document contient plusieurs sections sur la sélection de l'hyperviseur. Si vous lisez ce document de manière linéaire, vous ne devez pas charger le module KVM avant d'installer *nova-compute*.

Le service *nova-compute* dépend de *qemu-kvm*, qui installe le fichier */lib/udev/rules.d/45-qemu-kvm.rules*, permettant de définir les permissions correctes sur le périphérique */dev/kvm*.

Pour activer explicitement KVM, ajoutez les options de configuration suivantes dans le fichier */etc/nova/nova.conf* :

```
compute_driver = libvirt.LibvirtDriver
```

```
[libvirt]
```

```
virt_type = kvm
```

L'hyperviseur KVM prend en charge les formats d'image de machine virtuelle suivants :

- **Raw**
- **QEMU Copy-on-write (QCOW2)**

- **QED (Qemu Enhanced Disk)**
- **VMware virtual machine disk format (VMDK)**

Cette section explique comment activer KVM sur votre système. Pour plus d'informations, consultez la documentation spécifique à votre distribution :

- **Fedora** : Guide de démarrage de la virtualisation (documentation Fedora 22).
- **Ubuntu** : *KVM/Installation* (documentation communautaire Ubuntu).
- **Debian** : *Virtualization with KVM* (manuel Debian).
- **Red Hat Enterprise Linux** : Installation des paquets de virtualisation sur un système RHEL existant.
- **openSUSE** : Installation de KVM (manuel de virtualisation openSUSE).
- **SLES** : Installation de KVM (guide de virtualisation de SUSE Linux Enterprise Server).

Activation de KVM

Les sections suivantes décrivent comment activer la virtualisation matérielle KVM sur différentes architectures et plateformes. Pour effectuer ces étapes, vous devez être connecté en tant qu'utilisateur **root**.

Pour les systèmes x86

Vérifiez si votre processeur prend en charge les extensions de virtualisation *svm* (AMD) ou *vmx* (Intel) en exécutant la commande suivante :

```
grep -E 'svm|vmx' /proc/cpuinfo
```

1.
 - Si cette commande affiche un résultat, cela signifie que le CPU supporte la virtualisation matérielle.
 - Toutefois, même si la sortie est positive, vous devrez peut-être activer cette fonctionnalité dans le BIOS pour un support complet.
 - Si **aucun résultat** n'apparaît, vérifiez la documentation de votre système pour voir si votre CPU et votre carte mère prennent en charge la virtualisation.
 - Dans le BIOS, recherchez des options contenant **Virtualization**, **VT**, **VMX**, ou **SVM** et activez-les.

Vérifiez si les modules KVM sont bien chargés dans le noyau :

```
lsmod | grep kvm
```

- 2.

- Si la sortie contient **kvm_intel** (Intel) ou **kvm_amd** (AMD), alors les modules sont bien chargés.

Si aucun module KVM n'apparaît, chargez-le manuellement en exécutant :

```
modprobe -a kvm
```

-
- Chargez le module correspondant à votre processeur :

Intel :

```
modprobe -a kvm-intel
```

■

AMD :

```
modprobe -a kvm-amd
```

■

3. **Reconnectez-vous** après l'installation, car l'installation de KVM peut modifier les appartenances aux groupes utilisateurs.
4. Si les modules KVM ne se chargent pas automatiquement, suivez les procédures indiquées dans les sous-sections correspondantes.

Si les vérifications montrent que la virtualisation matérielle ou les modules noyau nécessaires sont désactivés ou absents, vous devrez soit **activer cette prise en charge sur votre système**, soit **trouver un autre système** qui la supporte.

Résumé et Explication

Ce document explique comment **installer et activer KVM (Kernel-based Virtual Machine)**, qui est l'hyperviseur par défaut pour OpenStack Compute.

Points clés à retenir

- **KVM est l'hyperviseur par défaut** pour OpenStack Compute.
- **Ne pas charger KVM trop tôt** lors de l'installation, car *nova-compute* dépend de *qemu-kvm*, qui configure les bonnes permissions.
- **Configurer KVM dans OpenStack** en modifiant `/etc/nova/nova.conf` avec les options adéquates.

- **KVM prend en charge plusieurs formats d'images VM**, comme QCOW2, RAW, et VMDK.
- **Vérifier la compatibilité matérielle** en s'assurant que le CPU supporte *svm* (AMD) ou *vmx* (Intel).
- **Activer la virtualisation dans le BIOS** si nécessaire.
- **Vérifier si les modules KVM sont bien chargés** et les charger manuellement avec `modprobe` si besoin.

En résumé, ce guide est une **procédure détaillée** pour **vérifier, configurer et activer KVM**, que ce soit pour un environnement OpenStack ou pour une simple virtualisation matérielle sur Linux.

COMMENT INTEGRER UNE BASE DE CONNAISSANCES A NOVA?

Il est possible d'étendre Nova pour y intégrer des mécanismes qui ressemblent à une base de connaissances afin d'influencer, voire de contrôler, certaines actions (comme la migration ou le placement des VMs) avant que Nova ne transmette l'ordre à KVM. Toutefois, ce n'est pas une fonctionnalité native, mais plutôt une personnalisation ou une extension de la plateforme OpenStack.

1. Intégration d'une "base de connaissances" dans Nova

Nova, en tant que composant de calcul d'OpenStack, dispose d'un scheduler hautement configurable qui détermine le placement des instances sur les hôtes physiques. Par défaut, le scheduler s'appuie sur des filtres (appelés « scheduler filters ») et des poids pour choisir le meilleur hôte en fonction de critères comme la charge, la disponibilité de ressources ou des métadonnées associées aux hôtes (via les *host aggregates* et *availability zones*).

Ce que cela implique pour l'intégration d'une base de connaissances :

- **Extensibilité via des filtres personnalisés :**
 Vous pouvez développer des filtres personnalisés qui, par exemple, interrogent une base de données ou un service externe (votre "base de connaissances") pour vérifier des caractéristiques spécifiques d'un hôte (architecture CPU, ressources disponibles, compatibilité réseau, etc.) avant d'autoriser une migration ou un placement de VM.
Exemple : Si vous avez une base de données contenant des informations détaillées sur chaque hôte (capacité de mémoire, version du CPU, etc.), un filtre personnalisé peut vérifier que, pour migrer une VM d'un hôte B vers un hôte A, ce dernier doit satisfaire à des critères précis (ex. : même type de

processeur, vitesse d'accès disque équivalente, etc.).

- **Utilisation des extra-specs et métadonnées :**

Vous pouvez aussi enrichir les *flavors* avec des extra-specs qui décrivent des exigences particulières ou enregistrer des informations détaillées sur chaque hôte. Ces informations peuvent être utilisées par le scheduler pour décider si une migration est envisageable ou si un hôte peut accueillir une VM donnée.

Exemple : Un flavor pourrait spécifier « compatibilité_migration = A_vers_B » et le scheduler vérifierait alors que les hôtes concernés possèdent cette caractéristique dans leur ensemble de métadonnées.

- **Orchestration avec d'autres services OpenStack :**

Des services comme Ceilometer ou Gnocchi, qui collectent des métriques et des informations sur l'infrastructure, pourraient être exploités pour alimenter votre base de connaissances et mettre à jour en temps réel l'état des hôtes. Un service middleware personnalisé pourrait alors interagir avec Nova pour déclencher ou bloquer certaines actions en fonction de ces données.

2. Migration de VMs en fonction des caractéristiques des machines

OpenStack propose des fonctionnalités de migration (live migration par exemple) de VM via Nova et KVM, mais la logique de décision quant à la faisabilité d'une migration se base principalement sur les ressources disponibles et la compatibilité de l'hyperviseur. Pour intégrer une étape d'analyse basée sur une base de connaissances, vous pourriez :

- **Analyser les caractéristiques préalablement enregistrées dans Nova :**

Avant de lancer une migration, un module personnalisé (ou une extension du scheduler) peut consulter la base de connaissances pour comparer les caractéristiques de l'hôte source (B) et de l'hôte destination (A). Par exemple, vérifier que les configurations matérielles (CPU, mémoire, capacités d'I/O) sont compatibles et que la migration ne risque pas de provoquer une incompatibilité ou une dégradation des performances.

- **Définir une logique de décision pour la migration :**

Vous pourriez implémenter une règle du type « migration autorisée uniquement de B vers A si :

- A dispose d'une capacité CPU équivalente ou supérieure.
- A a la même architecture ou des capacités de virtualisation identiques.

- Les configurations réseau et de stockage sont compatibles. »
Cette logique peut être codée dans un filtre ou dans une fonction de pré-validation qui s'exécute avant l'instruction de migration à KVM.
 - **Exemple pratique :**
Imaginons que vous avez deux hôtes :
 - **Hôte A :** CPU Intel Xeon de 3,0 GHz, 128 Go de RAM, support complet des extensions VT-x.
 - **Hôte B :** CPU Intel Xeon de 2,5 GHz, 64 Go de RAM, même support VT-x.Votre base de connaissances, alimentée en amont par des inventaires et des mesures de performance, indique que la migration d'une VM gourmande en ressources de B vers A est possible (grâce à la meilleure capacité de A) mais pas l'inverse. Un filtre personnalisé dans Nova pourra alors refuser une migration qui ne respecterait pas ces critères et autoriser uniquement celle de B vers A.
Ce contrôle préalable permet d'éviter des migrations qui pourraient aboutir à un déséquilibre de charge ou à des problèmes de compatibilité.
-

3. Comment procéder concrètement dans OpenStack ?

1. Développer des filtres personnalisés :

- Créez des filtres pour le scheduler de Nova qui consultent soit une base de données externe, soit les extra-specs et métadonnées associées aux hôtes et aux VMs.
- Intégrez ces filtres dans le pipeline du scheduler pour qu'ils s'exécutent avant de décider du placement ou de la migration d'une VM.

2. Enrichir le modèle de données de Nova :

- Utilisez les *host aggregates*, *availability zones* et extra-specs pour stocker des informations sur les caractéristiques matérielles et les contraintes.
- Vous pouvez aussi envisager d'étendre la base de données de Nova ou de connecter une base de connaissances externe via une API.

3. Mettre en place un service de monitoring :

- Intégrez Ceilometer ou un outil de monitoring personnalisé pour obtenir des informations en temps réel sur l'état des hôtes.
- Ces informations pourront être utilisées pour mettre à jour la base de connaissances et pour alimenter la logique décisionnelle du scheduler.

4. Test et validation :

- Avant de déployer en production, effectuez des tests approfondis pour vérifier que les règles de migration et de placement respectent bien les contraintes définies.
- Simulez des scénarios de migration pour voir si le système réagit comme prévu, par exemple en tentant des migrations inversées (A vers B) pour vérifier que le filtre bloque correctement les opérations non conformes.

Conclusion

En résumé, bien qu'OpenStack Nova ne propose pas par défaut une « base de connaissances » destinée à contrôler directement l'action de KVM, sa conception modulaire permet de développer et d'intégrer des extensions (via des filtres personnalisés, l'enrichissement de métadonnées, et l'utilisation d'outils de monitoring) pour introduire une logique avancée de prise de décision. Cela permettrait, par exemple, de vérifier si une migration de VM est viable d'un hôte à un autre (B vers A ou vice-versa) en fonction des caractéristiques matérielles préalablement définies dans Nova. Ce type de personnalisation demande un développement sur mesure et une bonne compréhension des mécanismes internes du scheduler de Nova ainsi que de l'écosystème OpenStack.

QUELLE EST LA SOLUTION LA PLUS FAISABLE ET COMMENT LA METTRE EN PLACE?

La solution la plus simple et efficace dans ce cas consiste à **étendre le scheduler de Nova avec un filtre personnalisé** qui s'appuie sur des métadonnées (extra-specs et host aggregates) pour décider du placement et de la migration des VM. Cette approche permet de tirer parti des mécanismes déjà existants dans OpenStack sans devoir mettre en place une infrastructure externe complexe pour votre « base de connaissances ».

Je vous propose la procédure suivante détaillée :

1. Contexte de l'environnement

- **Contrôleur** : Machine virtuelle hébergeant les services de Nova API, le scheduler et autres composants du contrôleur.

- **Noeud de calcul** : Machine virtuelle(s) qui héberge(nt) les instances et où KVM est installé.
- **Noeud Neutron** : Machine virtuelle dédiée à la gestion du réseau.

L'idée est de configurer le scheduler de Nova pour qu'il ne propose qu'un placement (ou qu'une migration) sur un hôte de calcul **compatible** avec les caractéristiques demandées par la VM (définies dans les extra-specs du flavor ou via des métadonnées stockées dans un host aggregate).

2. Préparer les métadonnées pour les hôtes et les flavors

a) Pour les hôtes (host aggregates)

Créer des agrégats d'hôtes

Sur le contrôleur, créez des agrégats qui regroupent les compute nodes par caractéristiques matérielles ou contraintes particulières.

Par exemple, pour deux groupes (disons « A » et « B ») :

```
nova aggregate-create aggregate_A zone_A
```

```
nova aggregate-create aggregate_B zone_B
```

1.

Ajouter des métadonnées aux agrégats

Attribuez une clé (ex. `migrate_direction`) pour indiquer la compatibilité.

Par exemple, pour l'agrégat A :

```
nova aggregate-update aggregate_A --metadata migrate_direction=A
```

Et pour l'agrégat B :

```
nova aggregate-update aggregate_B --metadata migrate_direction=B
```

2.

Ajouter les compute nodes aux agrégats

Par exemple, si votre compute node1 doit être dans A et compute node2 dans B :

```
nova aggregate-add-host aggregate_A <nom_compute_node1>
```

```
nova aggregate-add-host aggregate_B <nom_compute_node2>
```

3.

b) Pour les flavors (extra-specs)

Définir des extra-specs dans le flavor

Lors de la création ou modification d'un flavor, ajoutez une extra-spec pour définir la compatibilité de migration souhaitée.

Par exemple, pour autoriser uniquement une migration vers un hôte du groupe A, faites :

```
nova flavor-key <flavor_id> set migrate_direction A
```

1.

Ainsi, lorsqu'une VM sera créée avec ce flavor, elle portera cette exigence.

3. Créer un filtre personnalisé pour le scheduler de Nova

L'objectif est que le scheduler, lors du placement ou d'une demande de migration, vérifie que l'hôte de calcul possède bien la même valeur pour la clé `migrate_direction` que celle demandée par la VM (définie dans le flavor).

a) Écrire le code du filtre

Créez un fichier Python par exemple `custom_migration_filter.py`. Voici un exemple de code :

```
from nova.scheduler import filters
```

```
class MigrationCompatibilityFilter(filters.BaseHostFilter):
```

```
    """Filtre personnalisé pour vérifier la compatibilité de migration."""
```

```
    def host_passes(self, host_state, filter_properties):
```

```
        # Récupérer la valeur de l'extra-spec demandée dans le flavor
```

```
        flavor = filter_properties.get('instance_type', {})
```

```

required_direction = flavor.get('extra_specs', {}).get('migrate_direction')

if not required_direction:

    # S'il n'y a pas de contrainte, laisser passer

    return True


# Récupérer les métadonnées de l'hôte (stockées via host aggregates)

host_metadata = host_state.host_ram_free # par exemple, utilisez l'attribut
adapté

# Généralement, Nova fusionne les métadonnées des agrégats dans un
dictionnaire associé à l'hôte.

# On suppose ici qu'une clé "migrate_direction" a été fusionnée dans l'état de
l'hôte.

host_direction = host_state.get('migrate_direction')


if host_direction is None:

    # Si aucune info n'est trouvée sur l'hôte, on refuse par sécurité

    return False


# Vérifier la compatibilité

if host_direction == required_direction:

    return True

return False

```

Notes importantes :

- La manière d'accéder aux métadonnées d'un hôte peut varier selon la version d'OpenStack.

- Vous devrez vous assurer que les métadonnées des host aggregates sont bien propagées dans l'objet `host_state` ou trouver la méthode recommandée pour y accéder.

b) Installer le filtre sur le contrôleur

1. Placez le fichier `custom_migration_filter.py` dans un répertoire accessible par Python (par exemple, dans `/usr/lib/pythonX.Y/site-packages/nova/scheduler/filters/` ou dans un répertoire de votre choix auquel vous ajouterez le chemin).
2. Assurez-vous que le contrôleur (service nova-scheduler) peut importer ce module.

c) Modifier la configuration de Nova

Dans le fichier `/etc/nova/nova.conf` sur le contrôleur, modifiez la section du scheduler pour inclure votre filtre personnalisé dans la liste des filtres par défaut :

[scheduler]

Liste des filtres par défaut (ajoutez le nom complet de votre classe)

```
scheduler_default_filters = AvailabilityZoneFilter, RamFilter, DiskFilter,
ComputeFilter,
nova.scheduler.filters.custom_migration_filter.MigrationCompatibilityFilter
```

Remarque : Le chemin complet (module + classe) doit être correctement référencé.

3. Redémarrer le service nova-scheduler

```
sudo systemctl restart openstack-nova-scheduler
```

4. Validation et tests

a) Test de création d'instance

1. **Lancer une instance** avec un flavor qui a l'extra-spec `migrate_direction=A`.
2. Le scheduler devrait placer la VM sur un compute node appartenant à l'agrégat dont la métadonnée `migrate_direction` vaut `A`.

b) Test de migration

Pour tester la migration :

1. Tentez de migrer une VM depuis un hôte du groupe B vers un hôte du groupe A (ou inversement).
2. Votre filtre personnalisé doit vérifier la compatibilité et, si les valeurs ne correspondent pas, **refuser** ou **ne pas proposer** la migration.

5. Explications complémentaires

- **Pourquoi cette solution ?**

Elle s'appuie sur des fonctionnalités natives d'OpenStack (host aggregates, extra-specs, scheduler filters). Il n'est donc pas nécessaire de mettre en place une infrastructure externe pour stocker et interroger une base de connaissances.
- **Avantages**
 - **Simplicité** : Le développement d'un filtre personnalisé est relativement simple par rapport à l'implémentation d'un service externe complet.
 - **Intégration native** : Utilise la configuration et les mécanismes existants dans Nova, ce qui facilite la maintenance et la compréhension par d'autres administrateurs OpenStack.
- **Points à surveiller**
 - Assurez-vous que les métadonnées définies dans les agrégats soient bien propagées dans l'état des hôtes consulté par le scheduler.
 - Testez en environnement de pré-production avant de déployer en production pour éviter toute incompatibilité.

Conclusion

La mise en place d'un filtre personnalisé dans le scheduler de Nova, couplé à l'utilisation de host aggregates et extra-specs, constitue la solution la plus simple et efficace pour contrôler (et même restreindre) les migrations de VM en fonction des caractéristiques matérielles ou des contraintes prédéfinies. Cette approche exploite les mécanismes internes d'OpenStack et peut être déployée sur une architecture standard avec un contrôleur, des noeuds de calcul et un noeud Neutron. Les étapes ci-dessus détaillent le processus de configuration, le développement du filtre, ainsi que les tests nécessaires pour valider la solution.

JE VEUX POUVOIR RETRACER L'HISTORIQUE DU SCHEDULER: VOIR SI UNE MIGRATION A REUSSI OU SI CETTE DERNIERE A ECHOUÉ. ET JE VEUX POUVOIR SUIVRE EN TEMPS REEL COMMENT EVOLUE CETTE MIGRATION ET EVENTUELLEMENT VOIR SI IL Y' A MIGRATION OU PAS?

ET EVENTUELLEMENT D' AUTRES DETAILS UN PEU PLUS PRECIS

Pour surveiller le processus de migration via les logs du scheduler, il faut principalement s'assurer que le niveau de log est suffisamment verbeux (DEBUG) afin d'obtenir toutes les informations sur la prise de décision et le filtrage des hôtes. Voici comment procéder concrètement :

1. Configurer le niveau de log en DEBUG

a) Modifier la configuration du scheduler

Dans le fichier `/etc/nova/nova.conf` (sur le contrôleur où tourne le scheduler), vérifiez ou ajoutez les lignes suivantes :

```
[DEFAULT]
```

```
debug = True
```

```
[oslo_logging]
```

```
# Vous pouvez préciser le niveau de log global si nécessaire
```

```
log_level = DEBUG
```

b) Redémarrer le service du scheduler

Après modification, redémarrez le service pour appliquer les changements :

```
sudo systemctl restart openstack-nova-scheduler
```

2. Localiser et surveiller les logs du scheduler

Les logs du scheduler se trouvent généralement dans :

```
/var/log/nova/nova-scheduler.log
```

Pour les suivre en temps réel, utilisez par exemple :

```
tail -f /var/log/nova/nova-scheduler.log
```

Vous verrez alors apparaître des messages détaillant :

- **Les filtres appliqués** : Le scheduler passe en revue chaque filtre (par exemple, le filtre de compatibilité de migration) et loggue le résultat pour chaque hôte candidate.
 - **La sélection de l'hôte** : Une fois les filtres appliqués, le scheduler choisit l'hôte qui va recevoir l'instance ou sur lequel la migration sera effectuée. Le log affichera souvent quelque chose comme « Selected host: compute-XX for instance migration ».
 - **Les raisons de refus** : Si un hôte est rejeté, le log peut indiquer quel filtre n'a pas été satisfait (ce qui est utile si vous avez ajouté un filtre personnalisé).
-

3. Informations spécifiques à rechercher dans les logs

a) Messages de filtrage

Vous devriez voir des lignes indiquant que le scheduler applique des filtres. Par exemple :

```
DEBUG nova.scheduler.manager: Filtering hosts: [compute-01, compute-02, ...]
```

```
DEBUG nova.scheduler.filters: AggregateInstanceExtraSpecsFilter: host compute-01
passed filter migrate_direction=A
```

Si vous avez développé un filtre personnalisé, vous pouvez y ajouter des messages de log supplémentaires pour tracer précisément la décision, par exemple dans votre code :

```
import logging
```

```
LOG = logging.getLogger(__name__)
```

```
class MigrationCompatibilityFilter(filters.BaseHostFilter):
```

```
    def host_passes(self, host_state, filter_properties):
```

```
        flavor = filter_properties.get('instance_type', {})
```

```
        required_direction = flavor.get('extra_specs', {}).get('migrate_direction')
```

```
        if not required_direction:
```

```
            LOG.debug("Aucune contrainte de migration définie pour cette instance.")
```

```
            return True
```

```
        host_direction = host_state.get('migrate_direction')
```

```
        if host_direction is None:
```

```
            LOG.debug("L'hôte %s n'a pas de métadonnée migrate_direction, rejet de la
candidature.", host_state.host)
```

```
            return False
```

```
        if host_direction == required_direction:
```

```
            LOG.debug("L'hôte %s satisfait la contrainte migrate_direction=%s.",
host_state.host, required_direction)
```

```
            return True
```

```
        else:
```

```
            LOG.debug("L'hôte %s ne satisfait pas migrate_direction: attendu %s, trouvé
%s.", host_state.host, required_direction, host_direction)
```

```
return False
```

Cela vous permettra de suivre dans les logs comment le filtre se comporte pour chaque hôte candidat.

b) Messages de sélection et de migration

Lorsque le scheduler a terminé le filtrage, il enregistre généralement une ligne indiquant l'hôte choisi pour la migration. Par exemple :

```
INFO nova.scheduler.manager: Selected host compute-01 for instance  
<instance_id>
```

Cela vous indique que la migration (ou le placement initial) se fera sur l'hôte **compute-01**.

c) Suivi des tentatives de migration

Pour une migration, le processus est le suivant :

- **La demande de migration** est initiée via Nova (soit manuellement via la CLI, soit via une API).
- Le scheduler reçoit la demande et passe en revue les hôtes disponibles.
- Les logs détaillent quels hôtes passent ou échouent aux filtres (comme montré plus haut).
- Une fois le meilleur hôte identifié, le scheduler le note et transmet la décision au service nova-compute qui réalisera la migration.

Vous pouvez repérer les étapes de décision dans les logs, ainsi que d'éventuelles erreurs ou rejets de candidats.

4. Recueillir et analyser les informations

Pour résumer, voici comment vous pouvez recueillir les infos via les logs :

1. **Activez le mode DEBUG** pour obtenir des détails maximum.
2. **Surveillez en temps réel** le fichier
`/var/log/nova/nova-scheduler.log`.
3. **Cherchez les messages clés** :

- **Filtrage des hôtes** : Messages indiquant l'application des filtres et les critères (par exemple, `migrate_direction`).
 - **Sélection finale** : Message indiquant quel hôte a été sélectionné pour la migration.
 - **Messages de votre filtre personnalisé** (si vous en avez ajouté) pour comprendre pourquoi certains hôtes sont rejetés ou acceptés.
4. **En cas d'erreur ou d'échec** de migration, les logs vous donneront des indices sur la raison (métadonnées manquantes, incompatibilités, etc.).
-

Conclusion

En configurant le niveau de log à DEBUG et en ajoutant, si nécessaire, des messages de log dans vos filtres personnalisés, vous pourrez suivre en détail :

- **Si une migration a pu avoir lieu** (en observant les décisions du scheduler),
- **Si elle va se faire** (par le choix de l'hôte),
- **Comment elle est décidée** (en lisant la sortie des filtres appliqués).

Cela vous permettra de diagnostiquer et d'ajuster votre configuration pour garantir que les migrations se déroulent comme prévu dans votre environnement OpenStack.

PETIT COMMENTAIRE PAR RAPPORT A NOVA ET CINDER :

Oui, il est tout à fait possible de se passer de Cinder pour ce type de migration si votre objectif est d'utiliser une base de connaissances qui centralise d'autres critères (par exemple, les caractéristiques matérielles, la charge, la compatibilité des versions, etc.) pour décider si une migration est possible.

Explications :

- **Utilisation d'une base de connaissances personnalisée :**
Plutôt que de vous appuyer sur Cinder, qui est orienté vers la gestion des volumes de stockage, vous pouvez développer ou intégrer une base de connaissances contenant des informations détaillées sur vos hôtes et les configurations de vos VMs. Cette base pourrait stocker, par exemple, des informations sur :
 - Le type de CPU, la fréquence, le nombre de cœurs
 - La quantité de mémoire disponible et utilisée
 - Les extensions de virtualisation supportées (VT-x, AMD-V)

- Des contraintes spécifiques (version de firmware, compatibilité réseau, etc.)
- **Intégration dans Nova via des filtres personnalisés :**
 Vous pouvez développer un filtre personnalisé pour le scheduler de Nova qui, lors d'une tentative de migration, interroge cette base de connaissances afin de comparer les caractéristiques de l'hôte source et de l'hôte cible. Le filtre décidera alors, en fonction des règles que vous aurez définies, si la migration est autorisée ou non.
- **Exemple de scénario :**
 Supposons que votre base de connaissances indique qu'une migration de l'hôte B vers l'hôte A est possible uniquement si :
 - L'hôte A dispose d'une capacité CPU équivalente ou supérieure.
 - L'hôte A a une quantité de RAM suffisante.
 - Les configurations réseau et de virtualisation sont compatibles.
- Le filtre personnalisé pourra alors vérifier ces critères en consultant votre base de connaissances et, si tous les points sont validés, permettre à Nova de choisir l'hôte cible pour la migration.

Conclusion :

Vous n'êtes pas obligé d'utiliser Cinder si vos critères de décision ne concernent pas la compatibilité de stockage. Vous pouvez parfaitement mettre en œuvre une solution de migration basée sur une base de connaissances centralisée dans Nova, en développant des filtres personnalisés qui consultent les informations pertinentes pour décider de la faisabilité d'une migration.

En résumé, c'est une approche totalement viable et qui vous offre une grande flexibilité pour adapter la logique de migration aux spécificités de votre infrastructure.

QUESTION : DOIT ON ATTRIBUER LA MEMOIRE COMME CRITERES POUR EFFECTUER UNE MIGRATION DES VMs? SI OUI L'OUTIL CINDER DEVIENT UN MUST EN TERMES D'EFFICACITE, dois-je lui accorder de l'importance ? REPONSE NON

SEMAINE 4 RDV 06/03/2024:

je cherche a deployer Openstack sur grid 5000, en adaptant la procedure installation vue dans la play list suivante https://www.youtube.com/watch?v=mRXicKO4Rtl&list=PLHiVoTkxy-DdW6QN_NRVluBTqZcJgalK&index=2 a celle de l'outil grid5000,

Grace aux commandes fournies par caleb j'arrive a bien reserve des machines sur grid 5000 mais aussi grace aux commandes de mon compère jarrive a reserver une plage dadresses dans lesquelles les machines reservees doivent se situer et aussi pouvoir sauvegarder une session en cours avec toutes les installations et dependances deja presentes sur le disque .

Je pense pouvoir finir tout ce qui est dit plus haut dici demain apres midi, et peut etre par la meme occasion remplacer lhyperviseur kvm par xen

Pour le fichier de configuration envoyer par Caleb je regarderai cela dans la nuit du mardi au mercredi pour voir comment adapter a mon approche avec les metadonnees.

SEMAINE 5 RDV 13/03/2025

REFLEXION INTERESSANTE DE DEEPSEEK

Okay, let's try to figure out how to approach this problem. The user wants to migrate VMs between compute nodes in an OpenStack setup on Grid5000 using Xen as the hypervisor. They provided a JSON file that indicates whether migrations between specific clusters are possible based on CPU compatibility. The main requirement is to check if the migration from node A to node B is allowed by looking at the success flag in the JSON. If it's false, return the info message.

First, I need to understand the structure of the JSON file. It seems like each top-level key (like "dahu", "servan", etc.) represents a source cluster, and the nested keys under them are target clusters. For example, "dahu" has "abacus5", "grosminet", etc., each with a success boolean and an info message.

The user mentioned that the key check is whether the source cluster to target cluster entry in the JSON has success: true. If not, the migration should be blocked with the given info message. So the core logic is to read this JSON, check the source and target, and proceed accordingly.

Next, I need to think about how to integrate this check into the OpenStack migration process. OpenStack's Nova handles VM migrations, so perhaps a custom script or a Nova filter could be used to enforce this check. However, modifying Nova's code might be complex. A more straightforward approach might be to create a pre-migration script that consults the JSON file before allowing the migration command to proceed.

The user also provided installation steps for Xen and OpenStack via DevStack. It's important to ensure that the environment is correctly set up with Xen as the hypervisor and that live migration prerequisites (like shared storage and network configurations) are met.

For the implementation, the steps could be:

1. ****Parse the JSON File****: Read the provided JSON file to build a dictionary where each source cluster maps to allowed target clusters.

2. ****Determine Source and Target Clusters****: When a migration is requested, identify which clusters the source and target nodes belong to. This might require a mapping of node hostnames to their respective clusters.

3. ****Check Migration Feasibility****: Look up the source and target in the JSON. If success is false, output the info message and abort the migration. If true, proceed with the migration command.

4. ****Integration with OpenStack CLI****: Wrap the OpenStack migration command with a script that performs this check before executing the actual migration.

Potential challenges include accurately mapping node hostnames to clusters and ensuring the JSON file is accessible to the script. Also, handling cases where the source or target cluster isn't listed in the JSON.

Testing is crucial. The script should log its checks and outcomes, and there should be validation to ensure that all nodes are correctly categorized into clusters. The user mentioned that they might doubt their results, so thorough logging and possibly dry-run modes would be helpful.

Finally, documentation is important. The script should include comments explaining how it uses the JSON file, and the JSON file itself should be well-documented regarding which CPU features or vendors cause incompatibilities.

Putting this all together, the concrete implementation would involve a bash or Python script that acts as a middleware for the OpenStack migration command, enforcing the JSON-based rules.

Pour implémenter la migration des VMs avec vérification de compatibilité CPU via le fichier JSON fourni, voici une solution structurée :

1. Structure du fichier de compatibilité

Le fichier `webmail.grenoble-inp.org.txt` suit une structure hiérarchique :

```
```json
{
 "cluster_source": {
 "cluster_target": {
 "success": bool,
 "info": "raison d'échec"
 }
 }
}
```

```
}
}
...
```

### ### 2. Implémentation du script de migration

```
```bash
```

```
#!/bin/bash
```

```
# Script : openstack-migrate.sh
```

```
# Usage : ./openstack-migrate.sh <vm_id> <source_host> <target_host>
```

```
VM_ID=$1
```

```
SOURCE_CLUSTER=$2
```

```
TARGET_CLUSTER=$3
```

```
JSON_FILE="/chemin/absolu/webmail.grenoble-inp.org.txt"
```

```
# Récupération du statut depuis le JSON
```

```
check_migration() {
```

```
    jq -r --arg src "$SOURCE_CLUSTER" --arg dst "$TARGET_CLUSTER" \
```

```
    '[$src][$dst] // {"success": false, "info": "Combinaison cluster non documentée"}'  
    $JSON_FILE
```

```
}
```

```
# Exécuter la vérification
```

```
RESULT=$(check_migration)
```



```
SUCCESS=$(echo $RESULT | jq -r '.success')
```

```
INFO=$(echo $RESULT | jq -r '.info')
```

```
if [ "$SUCCESS" == "true" ]; then
```

```
    # Lancer la migration live avec OpenStack CLI
```

```
    openstack server migrate --live $TARGET_CLUSTER $VM_ID --wait
```

```
    echo "Migration réussie vers $TARGET_CLUSTER"
```

```
else
```

```
    echo "ERREUR: Migration impossible - $INFO"
```

```
    exit 1
```

```
fi
```

```
...
```

3. Intégration avec OpenStack

1. **Mapping des hôtes aux clusters** :

```
```bash
```

```
Fichier : /etc/node-clusters.csv
```

```
Format : hostname,cluster
```

```
econome-5,dahu
```

```
econome-9,abacus5
```

```
...
```

#### 2. \*\*Wrapper pour la CLI Nova\*\* :

```
```python
```

```
import csv
```

```
import json
```

```
import subprocess
```

```
def get_cluster(host):
```

```
    with open('/etc/node-clusters.csv') as f:
```

```
        return next(row[1] for row in csv.reader(f) if row[0] == host)
```

```
def migrate_vm(vm_id, target_host):
```

```
    source_host = get_vm_host(vm_id)
```

```
    source_cluster = get_cluster(source_host)
```

```
    target_cluster = get_cluster(target_host)
```

```
    with open('/chemin/webmail.grenoble-inp.org.txt') as f:
```

```
        data = json.load(f)
```

```
    if data.get(source_cluster, {}).get(target_cluster, {}).get('success', False):
```

```
        subprocess.run(f"openstack server migrate --live {target_host} {vm_id}",
shell=True)
```

```
    else:
```

```
        print(f"Blocked: {data[source_cluster][target_cluster]['info']}")
```

```
...
```

4. Déploiement sur Grid'5000

1. ****Placer le fichier JSON**** :

```
```bash
```

```
Sur tous les nœuds
```

```
sudo mkdir -p /etc/openstack/migration_rules

sudo cp webmail.grenoble-inp.org.txt /etc/openstack/migration_rules/

...
```

## 2. **\*\*Adapter la configuration Xen\*\*** :

```
``ini

Dans /etc/nova/nova.conf

[xenserver]

cpu_compare_mode = "strict"

xenapi_use_agent = false

...
```

### ### 5. Validation

Pour tester une migration :

```
``bash

Trouver l'ID de la VM

openstack server list

Lancer la migration avec vérification

./openstack-migrate.sh d155ccd2-01a1-489f-8164-5498d9872c1ab econome-5
econome-9

...
```

### ### 6. Justification des choix techniques

#### 1. **\*\*Emplacement du fichier JSON\*\*** :

- `/etc/openstack/migration\_rules` : Centralisé et accessible par tous les services
- Permissions : `root:nova 640` pour sécurité

## 2. **Méthode de vérification** :

- Utilisation de `jq` pour l'analyse JSON en bash
- Mapping statique hôte→cluster via fichier CSV pour flexibilité

## 3. **Intégration Xen** :

- Mode `cpu\_compare\_mode=strict` pour forcer la vérification CPU
- Désactivation de l'agent Xen pour éviter les conflits

## ### 7. Documentation complémentaire

Pour les cas non couverts par le JSON :

```
```bash
```

```
# Vérification manuelle des CPUs
```

```
xl info | grep -i cpu
```

```
virsh nodeinfo | grep -i cpu
```

```
# Comparer les flags
```

```
diff <(xenpm get-cpu-topology source) <(xenpm get-cpu-topology target)
```

```
```
```

Cette implémentation respecte les contraintes de Grid'5000 tout en intégrant les règles métier via le fichier JSON. Les vérifications CPU sont effectuées en pré-migration par Nova via le driver Xen.

D'accord, je dois aider l'utilisateur à résoudre son problème de migration de machines virtuelles entre différents clusters sur Grid5000 avec OpenStack et Xen. Le principal problème est que les nœuds de calcul sont dans des clusters différents, ce qui implique qu'ils ne sont pas nécessairement sur le même réseau, et cela bloque la migration via OpenStack en raison de la configuration réseau requise par Neutron.

Tout d'abord, je me souviens que pour qu'OpenStack fonctionne correctement, les nœuds de calcul doivent être sur le même réseau pour que Neutron puisse gérer les connexions réseau des VMs. Si les clusters sont différents, ils peuvent être sur des sous-réseaux ou VLANs distincts, ce qui empêche la communication nécessaire pour la migration live.

L'utilisateur a essayé d'installer Xen sur les nœuds et de configurer OpenStack avec DevStack, mais il rencontre des difficultés car les nœuds ne sont pas dans le même cluster, donc probablement pas dans le même réseau. Il mentionne également qu'il a suivi un guide d'installation de Xen sur OpenStack, mais que cela ne résout pas le problème de réseau entre clusters.

Je dois donc trouver une solution pour permettre la migration entre clusters tout en respectant les contraintes de Grid5000. Peut-être en utilisant un réseau privé virtuel ou en configurant des routes entre les clusters. Une autre possibilité est de réserver des machines dans des clusters différents mais sur le même VLAN en utilisant les fonctionnalités avancées de réservation OAR sur Grid5000.

Je me demande si Grid5000 permet de spécifier des contraintes de réseau lors de la réservation de machines, comme demander que les nœuds réservés soient sur le même VLAN. Si oui, cela résoudrait le problème en s'assurant que les nœuds, bien que dans des clusters différents, partagent le même réseau.

Ensuite, je dois vérifier comment configurer Neutron dans OpenStack pour gérer un tel réseau multi-clusters. Peut-être en utilisant des fournisseurs de réseau qui s'étendent sur plusieurs segments ou en configurant des tunnels réseau entre les nœuds.

De plus, l'utilisateur a fourni un fichier JSON qui indique la compatibilité des migrations entre clusters basée sur les CPUs. Il faut intégrer cette vérification dans le processus de migration pour s'assurer que les migrations ne sont tentées que si les clusters sont compatibles.

En résumé, les étapes à considérer sont :

1. **\*\*Réservation de nœuds sur le même VLAN\*\*** : Utiliser les options avancées d'OAR pour réserver des machines dans différents clusters mais sur le même réseau.
2. **\*\*Configuration réseau dans OpenStack\*\*** : Ajuster Neutron pour gérer un réseau étendu sur plusieurs clusters.
3. **\*\*Intégration de la vérification de compatibilité CPU\*\*** : Utiliser le fichier JSON pour valider les migrations avant de les exécuter.
4. **\*\*Tests et validation\*\*** : S'assurer que la migration fonctionne effectivement entre les clusters configurés.

Je dois maintenant structurer ces idées en une réponse claire et détaillée, en fournissant des commandes et configurations spécifiques pour chaque étape.

### ### \*\*Solution pour la Migration Multi-Clusters sur Grid'5000 avec OpenStack/Xen\*\*

---

#### #### \*\*1. Réserve de Nœuds dans des Clusters Différents mais sur le Même VLAN\*\*

Pour contourner la limitation réseau entre clusters, utilisez les **contraintes avancées d'OAR** pour forcer les nœuds à partager le même VLAN :

```
``bash
```

```
Réserver 2 nœuds de clusters différents mais sur le même VLAN
```

```
oarsub -t deploy -p "cluster='econome' or cluster='dahu" -l
/nodes=2,walltime=4:00:00 -t allow_classic_ssh=no -t vlantag=42 "sleep 4h"
```

```
...
```

- **`vlantag=42`** : Force l'utilisation d'un VLAN spécifique (remplacez `42` par un VLAN libre sur votre site Grid'5000).

- **Vérification du VLAN** :

```
``bash
```

```
cat /etc/network/interfaces | grep "vlan"
```

```
...
```

---

#### #### \*\*2. Configuration Réseau pour OpenStack Neutron\*\*

Adaptez la configuration Neutron pour gérer un réseau multi-clusters :

##### a. Sur le **Controller Node** :

```
```bash
```

```
sudo nano /etc/neutron/plugins/ml2/ml2_conf.ini
```

```
...
```

```
```ini
```

```
[ml2]
```

```
type_drivers = flat,vlan,vxlan
```

```
tenant_network_types = vxlan
```

```
mechanism_drivers = linuxbridge,l2population
```

```
[ml2_type_vlan]
```

```
network_vlan_ranges = physnet1:1000:2000 # Plage VLAN compatible avec
Grid'5000
```

```
[securitygroup]
```

```
firewall_driver = neutron.agent.linux.iptables_firewall.IptablesFirewallDriver
```

```
...
```

##### b. Sur les **Compute Nodes** :

```
```bash
```

```
sudo nano /etc/neutron/plugins/ml2/linuxbridge_agent.ini
```

```
...
```

```
```ini
```

```
[linux_bridge]
```

```
physical_interface_mappings = physnet1:eth0 # Interface physique partagée (A
REVOIR) //////////////////////////////////
```

```
[vxlan]
```

```
enable_vxlan = True
```

```
local_ip = <IP_LOCALE_DU_NŒUD> # IP interne du nœud
```

```
l2_population = True
```

```
...
```

```

```

#### \*\*3. Adaptation de la Configuration Xen pour les Migrations Multi-Clusters\*\*

Modifiez la configuration Nova pour autoriser les migrations entre clusters :

```
```bash
```

```
sudo nano /etc/nova/nova.conf
```

```
...
```

```
```ini
```

```
[DEFAULT]
```

```
cpu_shared_set = 0-63 # Adapter à votre architecture CPU (A EXPLIQUER)
```

```
allow_resize_to_same_host = False # Forcer la migration vers d'autres clusters
```



```
[xenserver]
```

```
live_migration_uri = "xenmigr://%s"
```

```
live_migration_inbound_addr = 0.0.0.0 # Accepter les migrations de n'importe où
```

```
...
```

```

```

#### #### \*\*4. Intégration du Fichier de Compatibilité CPU\*\*

Utilisez un script Python pour valider les migrations via le fichier JSON :

```
```python
```

```
# Script : validate_migration.py
```

```
import json
```

```
import sys
```

```
def check_migration(src_cluster, dst_cluster):
```

```
    with open('/etc/openstack/migration_rules.json') as f:
```

```
        data = json.load(f)
```

```
        return data.get(src_cluster, {}).get(dst_cluster, {}).get('success', False)
```

```
if __name__ == "__main__":
```

```
    src = sys.argv[1]
```

```
    dst = sys.argv[2]
```

```
    if check_migration(src, dst):
```

```

        print("Migration autorisée")

        sys.exit(0)

    else:

        print("Migration bloquée: Incompatibilité CPU")

        sys.exit(1)

...

- **Exécution avant migration** :

    ```bash

 openstack server migrate --live <dst_host> <vm_id> --pre-validation
 /usr/local/bin/validate_migration.py

 ...

5. Procédure Complète de Migration

    ```bash

    # Étape 1: Vérifier la compatibilité

    ./validate_migration.py dahu abacus5


    # Étape 2: Lancer la migration

    openstack server migrate --live compute-abacus5 vm_1234 --wait


    # Étape 3: Confirmer la migration

    openstack server resize --confirm vm_1234

```

'''

6. Résolution des Problèmes Courants

a. **Erreur "Network not reachable"**

- **Solution** : Vérifiez les routes entre clusters :

```
```bash
```

```
ip route show | grep <SUBNET_DESTINATION>
```

# Si nécessaire, ajoutez une route manuelle

```
sudo ip route add <SUBNET_DESTINATION> via <GATEWAY>
```

```
'''
```

### ##### b. \*\*Échec de Migration Live avec Xen\*\*

- \*\*Debug\*\* :

```
```bash
```

```
tail -f /var/log/nova/nova-compute.log | grep "xenapi"
```

Vérifiez que les versions Xen correspondent

```
xl info | grep xen_version
```

```
'''
```

7. Validation Expérimentale sur Grid'5000

1. ****Test de Connectivité**** :

```
``bash

# Depuis le nœud source

ping <IP_INTERNE_DESTINATION>

nc -zv <IP_DESTINATION> 16509 # Port XenAPI

...
```

2. ****Test de Performance**** :

```
``bash

iperf3 -c <IP_DESTINATION> -t 30 # Vérifiez le débit réseau (>1 Gbps requis)

...
```

****8. Documentation Complémentaire****

- ****Grid'5000 VLANs**** : [Advanced OAR Documentation](https://www.grid5000.fr/w/Advanced_OAR#Network_isolation)
- ****OpenStack Neutron**** : [Configuration multi-hosts](<https://docs.openstack.org/neutron/latest/admin/config-multi-host.html>)
- ****Xen Live Migration**** : [Xen Migration Guide](https://wiki.xenproject.org/wiki/Xen_Live_Migration)

Cette approche combine les spécificités de Grid'5000 (réservation VLAN) avec une configuration OpenStack/Xen adaptée, tout en intégrant vos règles métier via le fichier JSON. Les migrations ne seront déclenchées que si les clusters sont réseau-compatibles ****et**** CPU-compatibles.

APPROCHE MULTICOEURS

****Solution pour Réserver des Nœuds de Clusters Différents sur un Même Sous-Réseau****

****1. Correction de la Commande OAR****

La commande originale contient une erreur (`allow\_classic\_ssh=no` n'existe pas). Voici la syntaxe corrigée pour réserver 2 nœuds de clusters différents sur le même VLAN :

```
```bash
```

```
oarsub -t deploy \
```

```
-l "nodes=1,cluster='econome' + nodes=1,cluster='ecotype',walltime=4:00:00" \
```

```
-p "vlan=1" \
```

```
-t exotic \
```

```
"sleep 4h"
```

```
```
```

- ****Explication**** :

- `-l "nodes=1,cluster='econome' + nodes=1,cluster='ecotype'"` : Réserve 1 nœud du cluster `econome` et 1 du cluster `ecotype`.

- `-p "vlan=1"` : Active l'isolation réseau via un VLAN privé (obligatoire pour les clusters différents).

- `-t exotic` : Autorise des combinaisons non standard (nécessaire pour clusters différents).

2. Vérification du VLAN

Après réservation, vérifiez que les nœuds sont bien sur le même sous-réseau :

```
```bash
```

```
Sur chaque nœud
```

```
ip addr show vlan1 | grep "inet "
```

```
Doit afficher une IP dans le même sous-réseau (ex: 10.0.0.1/24 et 10.0.0.2/24)
```

```
```
```

```
---
```

3. Déploiement avec Kadeploy

Utilisez un environnement compatible VLAN (ex: `ubuntu2204-xen`) :

```
```bash
```

```
Déployer sur tous les nœuds réservés
```

```
kadeploy3 -e ubuntu2204-xen -f $OAR_NODE_FILE -k
```

```
```
```

```
---
```

4. Configuration OpenStack pour le VLAN Partagé

Dans `local.conf` de DevStack :

```
```ini
```

```
[[local|localrc]]
```

```
Activer Neutron avec VLAN
```

ENABLED\_SERVICES+=,neutron,q-agt,q-l3

Q\_AGENT=linuxbridge

Q\_ML2\_PLUGIN\_TYPE\_DRIVERS=vlan,flat

PHYSICAL\_NETWORK=public

PUBLIC\_INTERFACE=vlan1 # Interface VLAN créée par OAR

...

---

#### #### \*\*5. Migration de VMs Entre Clusters\*\*

Une fois OpenStack configuré :

```bash

Lancer une migration live

openstack server migrate --live <compute-dest> <vm-id> --os-compute-api-version 2.88

...

Documentation Complémentaire

- **Réservations Multi-Clusters** : [Advanced OAR - Combining Clusters](https://www.grid5000.fr/w/Advanced_OAR#Combining_clusters)

- **Isolation Réseau** : [Grid'5000 Network Isolation](https://www.grid5000.fr/w/Network_Isolation)

- **Environnements Kadeploy** : [Liste des environnements](https://www.grid5000.fr/w/Environments)

Validation Expérimentale

1. **Test Réseau** :

```
``bash
```

```
# Depuis le nœud source
```

```
ping <IP_VLAN_DESTINATION>
```

```
iperf3 -c <IP_VLAN_DESTINATION> -t 10 # Vérifiez le débit (>500 Mbps)
```

```
...
```

2. **Test OpenStack** :

```
``bash
```

```
openstack network agent list # Doit montrer les agents Neutron actifs sur les deux clusters
```

```
...
```

Cette méthode garantit que les nœuds de clusters différents communiquent via un VLAN privé, rendant la migration OpenStack/Xen possible.

EXEMPLE A PRENDRE AVEC PINCETTES

Solution Détaillée pour Résoudre l'Erreur OAR

1. Problème Identifié

L'erreur `cannot recognize the resource description: cluster='econome` indique une syntaxe incorrecte dans la spécification des ressources. La propriété `cluster` est valide, mais la structure de la commande OAR n'est pas conforme.

2. Commande OAR Corrigée

```
```bash
```

```
oarsub -t deploy \
```

```
-l "nodes=1,cluster='econome' + nodes=1,cluster='ecotype'" \
```

```
-l walltime=4:00:00 \
```

```
-t vlantag=1 \
```

```
-t exotic \
```

```
"sleep 4h"
```

```
```
```

- **Explications** :

1. **Syntaxe des ressources** :

Utilisez `+` pour combiner des ressources hétérogènes, et séparez les clauses `-l`.

```
```bash
```

```
-l "nodes=1,cluster='econome' + nodes=1,cluster='ecotype'"
```

```
-l walltime=4:00:00 # Déclaré séparément
```

```
```
```

2. **VLAN** :

Utilisez `-t vlantag=1` au lieu de `-p "vlan=1"` (la propriété `vlantag` est documentée [ici](https://www.grid5000.fr/w/Advanced_OAR#Network_isolation)).

3. **Option `exotic`** :

Obligatoire pour les réservations multi-clusters.

3. Vérification des Clusters Disponibles

Sur le site de Nantes, les clusters actifs sont :

- **econome** (Intel Xeon E5-2630v3)
- **ecotype** (Intel Xeon E5-2660v3)
- **dahu** (AMD Epyc 7351)

Consultez la liste complète via :

```
```bash
oarstat -f --json | jq '[]resources | .cluster' | sort -u
```
```

4. Validation du VLAN Après Réservation

Une fois les nœuds réservés, vérifiez le sous-réseau partagé :

```
```bash
Sur chaque nœud
ip addr show vlan1 | grep "inet"

Résultat attendu :

inet 10.1.1.2/24 brd 10.1.1.255 scope global vlan1
inet 10.1.1.3/24 brd 10.1.1.255 scope global vlan1
```
```

5. Exemple Complet avec Sortie

```bash

# Réreservation

\$ oarsub -t deploy -l "nodes=1,cluster='econome' + nodes=1,cluster='ecotype'" -l  
walltime=2:00:00 -t vlantag=1 -t exotic "sleep 2h"

[OAR] Job ID: 12345678

[OAR] Vous pouvez ajouter des nœuds avec : oarwalltime -j 12345678 +2h

# Connexion aux nœuds

\$ ssh econome-1

\$ ssh ecotype-3

# Vérification du VLAN

root@econome-1:~# ip addr show vlan1

2: vlan1@eth0: <BROADCAST,MULTICAST,UP,LOWER\_UP> mtu 1500 qdisc  
noqueue state UP

link/ether 00:16:3e:12:34:56 brd ff:ff:ff:ff:ff:ff

inet 10.1.1.2/24 brd 10.1.1.255 scope global vlan1

...

---

#### #### \*\*6. Documentation de Référence\*\*

- **\*\*Spécification des ressources OAR\*\*** :

[Advanced OAR - Combining Resources](https://www.grid5000.fr/w/Advanced\_OAR#Combining\_resources)

- **\*\*Isolation réseau\*\*** :

[Grid'5000 Network Isolation](https://www.grid5000.fr/w/Network\_Isolation)

- **\*\*Clusters de Nantes\*\*** :

[Hardware Nantes](https://www.grid5000.fr/w/Nantes:Hardware)

---

Cette solution garantit une réservation correcte de nœuds multi-clusters sur un VLAN partagé, en adéquation avec les contraintes techniques de Grid'5000.

SEMAINE 5 RDV 20/03/2025

Je cherche à réaliser la vérification de migration extra clusters (contours et détours) maintenant que Openstack a été bien déployée sur chaque nœud, que la procédure de vérification intra cluster est un peu mieux réalisée je vois la faisabilité du truc sur les clusters extra., raffiner le processus de migration initialement décrit, raffiner l'intégration de XEN dans openstack qui avait déjà été fait précédemment,

Et aussi se rassurer que tout jusqu'ici ait été bien documenté, et est cohérent avec l'évolution suivie

Bien évidemment faire un powerpoint global de tout ceci,

Appréter la semaine du rapport

**Also, remind them to check storage compatibility. Even if networks are okay, if storage isn't shared and block migration isn't enabled, it won't work. Their JSON needs to include storage details.**

**IMPORTANT Grid'5000 est une plateforme de recherche expérimentale qui regroupe des machines réparties sur plusieurs sites et organisées en clusters. Voici quelques points pour répondre à vos questions :**

### **1. Stockage partagé**

- Au sein d'un même cluster et d'un même site :  
Le stockage sur Grid'5000 n'est pas systématiquement partagé entre tous les nœuds d'un cluster. Par défaut, les nœuds disposent généralement de leur stockage local. Toutefois, Grid'5000 offre aussi des services de stockage mutualisé (par exemple via des espaces de stockage accessibles en NFS ou des solutions spécifiques comme le Group Storage) qui permettent de partager des données entre plusieurs nœuds.
- Entre clusters :  
Il n'existe pas de stockage partagé automatiquement entre des nœuds appartenant à des clusters différents sur un même site, sauf si vous configurez explicitement un espace de stockage partagé accessible par ces nœuds (via une réservation d'espace commun par exemple).

## 2. Réseau physique et VLAN

- Même réseau physique ?  
Dans un site Grid'5000, les nœuds d'un cluster sont généralement interconnectés par le réseau local du site (souvent via Ethernet ou des réseaux hautes performances comme Infiniband). Cependant, ils ne se trouvent pas forcément dans le même sous-réseau IP par défaut.
- VLAN :  
Grid'5000 propose des mécanismes comme KaVLAN, qui permettent de réserver et de configurer des VLANs dédiés pour isoler les nœuds d'une expérience. En réservant des ressources via OAR et en utilisant KaVLAN, vous pouvez garantir que les nœuds réservés (même issus de clusters différents sur un même site) se trouvent dans le même domaine de diffusion (même VLAN). Ceci est crucial pour des applications comme la migration de VMs via Xen/OpenStack, où un réseau de migration dédié est nécessaire.

### En résumé :

- Stockage : Les nœuds d'un même cluster partagent souvent leur stockage local, mais un stockage partagé n'est disponible que si vous le demandez et le configurez explicitement (et cela est généralement limité au sein d'un même cluster ou site).
- Réseau : Par défaut, les nœuds dans un même site sont interconnectés par l'infrastructure réseau du site, mais pour isoler vos expériences (et garantir une même diffusion pour la migration par exemple), vous devez réserver un VLAN dédié (via KaVLAN).
- Pour vos besoins de migration, vous devrez donc créer un VLAN dédié (en réservant via KaVLAN) pour que tous les nœuds réservés, même issus de clusters différents, se trouvent dans le même domaine de diffusion.

Ces informations sont conformes aux principes décrits dans la documentation Grid'5000 sur la réservation de ressources, l'utilisation des espaces de stockage mutualisés, et la configuration des VLANs via KaVLAN

### Raisonnement concernant l'approche réseau :

La réponse est oui, c'est possible, mais cela demande une configuration soignée. En effet, pour qu'une migration (qu'elle soit live ou par bloc) fonctionne dans un environnement Xen via

OpenStack, il faut que les hôtes (même s'ils n'ont pas de stockage partagé) puissent communiquer sur un réseau dédié pour la migration.

Si vos nœuds se trouvent sur des sous-réseaux différents et qu'il n'existe pas de routage ou de pontage entre eux, vous pouvez créer un réseau VLAN dédié et y tagger les interfaces de migration sur chacun des hôtes. Cela permettra de créer un domaine de diffusion commun dans lequel la migration pourra s'effectuer, car Xen (via la commande « xl migrate », par exemple) nécessite que le transfert de mémoire et d'état se fasse sur un réseau fiable et accessible par les deux hôtes.

Quelques points importants :

- **Réseau de migration dédié :** Vous devez configurer vos interfaces réseau sur chaque nœud pour qu'elles rejoignent ce VLAN (par exemple, avec un tag spécifique sur les ports physiques ou via une interface virtuelle dédiée). Cela garantit que le trafic de migration passe sur un chemin réseau commun.
- **Migration sans stockage partagé :** Si vos hôtes ne partagent pas le stockage, vous devrez utiliser la migration par bloc (block migration) qui transfère aussi les disques via le réseau. Cela peut impliquer une interruption plus longue et plus de charge réseau, mais reste faisable tant que le réseau VLAN est performant.
- **Configuration de Xen :** Vérifiez que la configuration de Xen (dans le fichier de configuration ou via la commande « xl migrate ») spécifie bien l'interface de migration ou l'adresse IP correspondant à ce VLAN, afin que la connexion soit établie sur ce domaine.

En résumé, en créant un réseau VLAN dédié reliant vos nœuds et en configurant correctement les interfaces sur ce VLAN, vous contournez l'absence de routage/bridging entre vos sous-réseaux d'origine, ce qui rend la migration possible même avec la librairie Xen et sans stockage partagé.

Cette approche est souvent utilisée dans des environnements expérimentaux comme Grid5000 où l'on peut réserver des ressources réseau spécifiques pour créer des domaines de migration cohérents.

## **FINALISATION DU PROCESSUS DE MIGRATION INTRACLUSTER**

### **Vérification du réseau entre clusters (même site ≠ même réseau)**

- **Fait technique :** Deux clusters d'un même site ne partagent pas nécessairement le même réseau. Sur GRID5000, chaque cluster peut avoir son propre sous-réseau, même au sein d'un même site, et dans le cas où ce n'est pas le cas dans la fiche technique on a abordé comment cela pouvait être changé dans les deux cas, pour vérifier cela on fait la commande suivante:

- Comment vérifier :

# Lister les réseaux disponibles et leurs associations aux clusters

```
openstack network list --long | grep -E "Name|Nantes"
```

```
openstack subnet list --long | grep "Nantes"
```

Si les clusters econome et ecotype partagent un réseau commun (ex : physique ou vlan-xxx), la migration directe est possible.

Sinon, il faut une connectivité réseau inter-sous-réseaux (vérifier avec les administrateurs de GRID5000).

### Vérification Réseau entre Clusters

Objectif : Confirmer que les clusters econome et ecotype partagent un réseau commun ou peuvent router le trafic.

a. Liste des réseaux OpenStack :

```
openstack network list --long | grep -E "Nantes|Name"
```

# Exemple de sortie :

```
| physnet1_nantes | Nantes (physique) |
```

```
| vlan-42 | econome-private |
```

```
| vlan-37 | ecotype-private |
```

b. Vérifier la connectivité intra -VLAN :

# Sur une VM du cluster econome :

```
ping -c 4 10.37 # IP d'une VM sur ecotype-private
```

# Si échec, vérifier les routes :

```
openstack router list | grep nantes
```

```
openstack router show <router_id> -c routes
```

# Doit inclure : "destination": "10.37.0.0/24", "nexthop": "10.42.0.1"

### ATTENTION:

Un autre moyen aurait été de faire un pont physique pour que les machines puissent communiquer et ainsi être en contact( il faudrait déjà qu'elles aient un port en commun , tout est détaillé dans le fichier Réseau physique( qui dit comment faire cela)

AAAA

## 2. Mise à jour de votre base de connaissances (CSV + JSON)

### a. Fichier CSV (`clusters_network.csv`) :

Ajoutez une colonne **Réseau** pour mapper les clusters aux réseaux :

```
Cluster,Site,Xen_Version,PV_drivers,Réseau,Block_Migration
econome,Nantes,4.16.1,xen-blkfront/xen-netfront,vlan-42,Yes
ecotype,Nantes,4.16.1,xen-blkfront/xen-netfront,vlan-37,Yes
```

Vu QUON TRAVAILLE AVEC XEN voila ce quil faut savoir dessus par rapport a notre probleme specifique

### Spécificités de Xen sur GRID5000/OpenStack

- Paravirtualisation (PV) : Les VMs sous Xen sont généralement en mode *paravirtualisé* (pas de hardware virtuel), ce qui impose des compatibilités strictes entre hyperviseurs source et destination.
- Stockage : Xen nécessite un stockage partagé (ex: Ceph) pour les migrations live, ou une *block migration* (copie des disques via le réseau).
- Réseau : Les interfaces réseau des VMs Xen (frontend/backend) doivent être compatibles entre clusters.

## 2. Vérifications critiques pour Xen

### a. Compatibilité des hyperviseurs :

```
bash
```

```
Copy
```

# Sur les nœuds source et destination, vérifier la version de Xen et le dom0

```
ssh "xl info | grep -E 'xen_version|cc_compiler'"
```

```
ssh ecotype-node "xl info | grep -E 'xen_version|cc_compiler'"
```

- Les versions de Xen et compilateurs doivent correspondre (ex: Xen 4.8 + GCC 7.5).

### b. Connectivité Xen-Xen :

```
bash
```

```
Copy
```



# Tester la connectivité entre hyperviseurs (port 8002 par défaut pour libvirt/xen)

nc -zv 8002 # Depuis ecotype-node

nc -zv ecotype-node 8002 # Depuis econome-node

- Si échec, vérifiez les règles iptables/security groups d'OpenStack et du réseau physique.

### c. Configuration Nova pour Xen :

bash

Copy

# Vérifier le driver libvirt et le type de virtualisation sur les nœuds

| grep -E "driver|type"

openstack hypervisor show ecotype-node | grep -E "driver|type"

- Doit retourner driver=libvirt et type=xen.

S'il ya un probleme avec Openstack et que cela echoue on peut faire un zoom sur lanalyse du débogage avec Xen

en executant depuis un noeud un source( noeud de calcul) la commande suivante: Si OpenStack échoue, testez directement avec Xen :

# Depuis le nœud source (ecotype-node)

xl migrate -C \$INSTANCE\_NAME econome-node

- -C : Vérifie la compatibilité avant migration (*nécessite un DNS/SSH configuré entre nœuds*).

### Petite Reflexion importante pour la suite :

Okay, let me try to wrap my head around this. The user is working on migrating VMs between different clusters on Grid5000 using OpenStack and Xen. They've had previous discussions where they set up a JSON-based compatibility check using jq and CSV files. Now, they want to know how to handle migrations both within the same cluster and between different clusters (econome and ecotype in Nantes), considering network compatibility and the existing JSON validation.

First, I need to recall the previous setup. They used a JSON file to determine if migration is possible based on CPU compatibility and other factors. The JSON is read via jq, and there's a CSV mapping clusters to attributes. The key now is to ensure network compatibility between clusters, as being on the same site (Nantes) doesn't automatically mean the same network.

The user is asking specifically about reserving control and compute nodes on different clusters and ensuring the network allows migration. They want to integrate this network check into their existing JSON/CSV validation system.

So, the main points are:

1. **Network Compatibility**: Even within the same site, clusters might be on different subnets/VLANs. Need to check if VMs in different clusters can communicate over the network.
2. **Integration with Existing Checks**: They already have a process using JSON and jq to validate migrations. Need to add network checks to this.
3. **Xen Hypervisor Considerations**: Since they're using Xen, there might be specific requirements for live migration, like shared storage or specific network configurations.

Let me break down the steps they need to take:

**1. Verify Network Configuration:**

- Check if the clusters (economy and ecotype) are on the same VLAN or subnet.
- Use OpenStack CLI to list networks and subnets associated with each cluster.
- If they're on different networks, check if there's routing between them or if they can be bridged.

**2. Update CSV and JSON Files:**

- Add network information to the CSV file, specifying which VLANs/subnets each cluster uses.
- Modify the JSON schema to include network compatibility in addition to CPU compatibility.

**3. Enhance the Validation Script:**

- Use jq to parse both CPU and network compatibility from the JSON/CSV.
- If networks are different but routed, ensure that the script acknowledges this (maybe with a warning or specific flag).

**4. Xen-specific Checks:**

- Ensure that Xen's live migration requirements are met, like shared storage (e.g., Ceph) or block migration settings.
- Verify that the Xen hypervisors on both clusters are configured to allow migrations (libvirt settings, firewall rules).

**5. Testing:**

- Before actual migration, test network connectivity between VMs on different clusters.
- Use tools like ping, iperf, or OpenStack's network test commands to validate latency and bandwidth.

### **\*\*Potential Pitfalls:\*\***

- Assuming same site implies same network, which might not be true.
- Overlooking firewall rules or security groups in OpenStack that block inter-cluster communication.
- Xen configuration differences between clusters (e.g., different versions or CPU flags).

### **\*\*Integration with Nova:\*\***

- The user mentioned integrating this into Nova's decision-making. This might involve creating a custom scheduler filter that checks both the JSON/CSV data and network compatibility before allowing migration.

### **\*\*Documentation References:\*\***

- OpenStack CLI commands for network and subnet listing.
- Xen documentation on live migration prerequisites.
- Grid5000's network architecture details for Nantes site.

Putting this all together, the user needs a comprehensive script that checks all compatibility layers (CPU, network, Xen config) using their existing JSON/CSV setup, then proceeds with migration only if all conditions are met. The script should output clear reasons if migration is blocked, helping the user troubleshoot issues like network misconfiguration or missing CPU features.

### **Configuration Nova :**

**Créez un filtre custom pour Nova :**

**a. Fichier Python (xen\_compatibility\_filter.py) :**

**l'ajouter a Nova de la facon suivante : # /etc/nova/nova.conf**

**scheduler\_default\_filters=...,XenCompatibilityFilter**

**Pour deployer**

**kadeploy3 -e ubuntu2204-xen -k**

**./stack.sh # DevStack avec config réseau adaptée**

### **FAQ Technique**

**Q : Pourquoi utiliser x1 info plutôt que virsh nodeinfo ?**

**R : Xen utilise x1 comme outil natif, qui donne des informations plus précises sur la paravirtualisation.**

Si cela est nécessaire et se rassurer que la mise en commun dans un même VLAN a bien marché, ou même la mise en commun via un bridge est ok( dépendamment du moyen utilisé)

Q : Comment auditer les règles de routage OpenStack ?

R : Utilisez :

```
neutron router-port-list <router_id> | grep "10.37.0.0/24"
```

### Petite Reflexion sur la modification du fichier Wrapper Cli

Okay, let's see what the user is asking here. They provided a Python script that acts as a wrapper for the OpenStack CLI to handle VM migrations between clusters based on a JSON compatibility file. They want to modify this script to incorporate the previous discussions about Xen hypervisor considerations, network checks, and other validations discussed in the thread.

First, I need to recall the key points from the previous solutions. The user wants to ensure that migrations are only allowed if the source and target clusters are compatible in terms of CPU, network (same VLAN or routed), Xen versions, and other factors. The existing script checks the JSON file for a 'success' flag, but now needs additional checks.

The original script reads the source and target clusters from a CSV, checks the JSON, and runs the migration command. The modifications needed include:

1. **Xen Compatibility Check**: Verify that the source and target hypervisors are running the same Xen version and have compatible configurations.
2. **Network Validation**: Ensure that the source and target clusters are on the same VLAN or have proper routing set up.
3. **Error Handling and Logging**: Improve error messages and logging for better debugging.
4. **Dry Run Option**: Allow testing the migration without executing it.
5. **Mapping VM to Host**: The existing code has a function `get_vm_host` which isn't defined. Need to implement that to get the current host of the VM.
6. **Security Considerations**: Ensure the script handles sensitive data securely, especially when dealing with shell commands.

Let's go step by step.

#### **1. Implementing `get_vm_host` Function:**

The original code references `get_vm_host(vm_id)` but doesn't define it. This function should use OpenStack CLI to fetch the current host of the VM. Using `openstack server show` with the appropriate fields.

#### **2. Xen Version Check:**

Need to retrieve Xen version from both source and target hypervisors. This can be done via SSH or using OpenStack hypervisor details. Since OpenStack might not expose Xen version directly, perhaps using ``xl info`` over SSH.

### **\*\*3. Network Check:\*\***

Check if the source and target clusters are on the same network. This can be derived from the CSV file (if network info is included) or by querying OpenStack's network details.

### **\*\*4. Modifying the CSV and JSON Structure:\*\***

The CSV should include additional columns like 'VLAN' or 'Network'. The JSON might need more detailed compatibility info, not just a boolean 'success'.

### **\*\*5. Dry Run Mode:\*\***

Adding a command-line flag (like ``--dry-run``) to simulate the checks without performing the migration.

### **\*\*6. Error Handling:\*\***

Using try-except blocks to catch exceptions, especially for subprocess calls and file operations.

### **\*\*7. Logging:\*\***

Implement logging to a file or stdout for audit purposes.

### **\*\*8. Security:\*\***

Avoiding `shell=True` in `subprocess.run` to prevent shell injection vulnerabilities. Using parameterized commands instead.

Putting this all together, the modified script would include these new checks, fetch necessary data from OpenStack and system commands, and provide comprehensive validation before proceeding with the migration.

### **3. Améliorations principales par rapport a l'ancienne structure:**

- Vérification multi-couches (Xen, réseau, CPU, règles JSON)
- Logging détaillé avec rotation automatique
- Mode dry-run pour simulation
- Gestion d'erreurs renforcée
- Interface CLI paramétrable

Pour pouvoir compiler le fichier Wrapper cli pour la migration exta cluster il faut installer les dependances suivantes:

```
sudo apt-get install python3-openstackclient jq
```

```
pip install argparse logging
```

## Utilisation du Wrapper Cli

### # Mode normal

```
python3 openstack_wrapper.py <vm_id> <target_host>
```

### # Mode simulation

```
python3 openstack_wrapper.py <vm_id> <target_host> --dry-run
```

Cette version implémente une vérification en 4 étapes :

1. Règles métier via JSON
2. Compatibilité Xen (version + compilateur)
3. Compatibilité réseau (VLAN + routage)
4. Compatibilité CPU (flags processeur)

Pour revenir au niveau reseau , voila les ameliorations faites par rapport a l'ancien modele existant :

2. Vérifications multicouches :
  - Réseau (VLAN direct ou routé)
  - Compatibilité Xen (version + compilateur)
  - Flags CPU
  - Règles métier JSON

Si on veut pouvoir tester le fichier à exécuter au niveau du controller pour pouvoir executer la migration, il faut executer dans le terminal utilisateurrr

### # Mode dry-run

```
./openstack-migrate.sh d155ccd2-01a1 econome-node3 --dry-run
```

### # Journalisation complète

```
tail -f /var/log/openstack_migration.log
```

### # Sortie détaillée

```
[2023-10-20 14:32:15] ERREUR: Flag CPU manquant (pdpe1gb)
```

Et il faut pouvoir avoir les dependances suivantes installées :

```
sudo apt-get install jq sshpass
```

```
chmod 600 /etc/node-clusters.csv
```

COTE RESEAU ET CONDITIONS DE MIGRATIONSTOUT EST OKAY:

MAINTENANT CONCERNANT LA MIGRATION EN ELLE-MEME

Voila la ce quil faut verifier au fur et a mesure :

Voici une proposition détaillée pour mettre en place ces deux aspects dans un déploiement OpenStack sur Grid5000, en prenant en compte que tous les nœuds (contrôleur et calcul) se trouvent dans le même VLAN et que l'hyperviseur utilisé est Xen :

---

## 1. Mise en place du stockage NFS

But :

Partager les images (par exemple pour Glance ou pour que Nova puisse accéder aux disques de VM) depuis le contrôleur vers tous les nœuds de calcul.

Étapes à suivre :

1. Installer et configurer le serveur NFS sur le nœud de contrôle :

- Installez le paquet NFS (ex. sur Ubuntu : `sudo apt install nfs-kernel-server`).
- Choisissez le répertoire à partager, par exemple `/var/lib/glance/images` ou un répertoire dédié aux images.

Modifiez le fichier `/etc/exports` pour y ajouter une ligne du type :

```
/var/lib/glance/images *(rw,sync,no_root_squash)
```

- Cela permet à tous les clients du VLAN d'accéder au partage en lecture/écriture.

Rechargez la configuration du serveur NFS avec la commande :

```
sudo exportfs -ra
```

○

2. Configurer les nœuds de calcul pour monter le partage NFS :

- Installez le client NFS (ex. `sudo apt install nfs-common`).

- Créez un point de montage local sur chaque nœud, par exemple `/mnt/nfs/images`.

Montez le partage en utilisant une commande du type :

```
sudo mount <IP_du_controller>:/var/lib/glance/images /mnt/nfs/images
```

○

Pour automatiser ce montage au démarrage, ajoutez une entrée dans `/etc/fstab` sur chaque nœud :

```
<IP_du_controller>:/var/lib/glance/images /mnt/nfs/images nfs defaults 0 0
```

○

### 3. Vérification et contraintes réseau :

- Assurez-vous que le VLAN dans lequel se trouvent tous les nœuds autorise le trafic NFS (les ports TCP/UDP 2049 notamment).
- Si un pare-feu est en place, autorisez ce trafic entre les nœuds et le contrôleur.

---

## 2. Utilisation de Kadeploy avec un environnement Xen préinstallé

Contexte et objectif :

Sur Grid5000, Kadeploy est souvent utilisé pour déployer des environnements de test sur les nœuds. Ici, il s'agit de passer d'un environnement antérieur (peut-être basé sur une image standard) à une image spécialement préparée pour utiliser Xen (par exemple, une image nommée `ubuntu2204-xen`).

Ce qu'il faut faire :

### 1. Préparation ou obtention de l'image compatible Xen :

- Assurez-vous que l'image `ubuntu2204-xen` intègre déjà les paquets et la configuration nécessaires pour démarrer et gérer l'hyperviseur Xen (les modules, la configuration du démarrage, etc.).
- Vous devez vérifier que cette image est compatible avec le matériel et la configuration réseau de Grid5000. En général, Grid5000 permet ce type de déploiement, mais il faut parfois adapter l'image en fonction des spécificités des nœuds (drivers, firmware...).

### 2. Modification du fichier de déploiement Kadeploy :



- Dans votre fichier de configuration Kadeploy, modifiez la référence à l'image à déployer afin d'utiliser **ubuntu2204-xen** au lieu de l'ancienne image.
- Vérifiez que toutes les options spécifiques à Xen (par exemple, la présence du démon **xenstored**, la configuration de **xl** ou **xen-api**) sont bien activées dans le script de déploiement.

### 3. Déploiement sur Grid5000 :

- Lancez la commande de déploiement via Kadeploy sur les nœuds souhaités.
- Pendant le déploiement, Kadeploy va restaurer l'image sur chaque nœud. Comme tous les nœuds font partie du même VLAN, la communication réseau doit être fluide pour permettre la synchronisation de l'environnement et le transfert des images (notamment si le stockage partagé NFS est utilisé pour certains services d'OpenStack).

### 4. Vérification post-déploiement :

- Une fois l'environnement déployé, vérifiez que Xen est bien démarré et opérationnel sur chaque nœud.
- Testez le fonctionnement de l'hyperviseur en créant une VM par exemple via Nova, en vous assurant que le driver Xen (souvent basé sur XenAPI) est correctement utilisé.

#### Remarque sur Grid5000 :

Grid5000 est conçu pour des expérimentations et déploiements de ce type. Ainsi, il est tout à fait possible d'utiliser Kadeploy pour déployer un environnement Xen. Il suffit de s'assurer que l'image utilisée (ici **ubuntu2204-xen**) est correctement préparée et que les nœuds disposent des droits nécessaires pour charger et exécuter Xen. Vous pouvez consulter la documentation Grid5000 et Kadeploy pour obtenir des exemples de fichiers de configuration adaptés et vérifier les prérequis matériels et logiciels.

---

## Conclusion

#### Pour récapituler :

- **Stockage NFS :**  
Vous installez et configurez un serveur NFS sur le contrôleur afin de partager un répertoire (contenant par exemple les images Glance). Chaque

nœud de calcul montera ce partage via une configuration locale (avec des entrées dans `/etc/fstab` pour automatiser le montage). Assurez-vous que la configuration réseau (VLAN, firewall) autorise le trafic NFS.

- **Kadeploy avec environnement Xen :**  
Il est possible de déployer via Kadeploy un environnement avec Xen préinstallé (tel que `ubuntu2204-xen`). Il vous faut adapter vos fichiers de déploiement pour pointer vers cette image et vérifier que l'image est compatible avec l'infrastructure Grid5000. Ce changement est impératif par rapport à l'ancienne configuration qui n'utilisait peut-être pas Xen.

Ces configurations permettront à OpenStack de fonctionner correctement dans l'environnement Grid5000 avec Xen, en s'assurant que le partage des images via NFS est accessible à tous et que le déploiement via Kadeploy installe un environnement adapté pour l'hyperviseur Xen.

citeturn0search0

## DEROULEMENT GLOBAL DE LA MIGRATION :

Voici un exemple détaillé de la façon dont se déroule, étape par étape, une migration de machine virtuelle (VM) dans OpenStack – en supposant que la phase de vérification préalable (état de l'instance, compatibilité, ressources, connectivité réseau, etc.) a déjà été validée – dans un environnement utilisant Xen comme hyperviseur. Cette description tient compte du rôle de Nova et des autres composants OpenStack impliqués, ainsi que de l'impact sur l'image (le support disque) de la VM.

---

### 1. Rôle de Nova et préparation de la migration

#### a. Initiation via Nova :

- **Commande de migration :** L'administrateur (ou une orchestration automatisée) lance la commande de migration (par exemple, via `nova migrate` pour une migration à froid ou `nova live-migrate` si la configuration et l'infrastructure Xen le permettent).
- **Sélection du nœud de destination :** Nova, via son ordonnanceur (scheduler), sélectionne un nœud de calcul de destination qui dispose de ressources suffisantes et qui supporte le même hyperviseur (ici, Xen) avec une configuration compatible.

## b. Composantes impliquées :

- **Nova Compute** : Sur le nœud source et sur celui de destination, le service nova-compute gère l'interaction avec l'hyperviseur. Dans le cas de Xen, Nova utilisera le driver XenAPI (ou un autre driver adapté) pour communiquer avec l'hyperviseur.
  - **Keystone** : Pour l'authentification et l'autorisation dans l'ensemble du processus.
  - **Neutron** : Pour s'assurer que la connectivité réseau (IP, routage, sécurité) est maintenue sur le nouvel hôte.
  - **Cinder / Glance** : Selon que la VM utilise des volumes persistants ou des images pour son disque système. Pour les déploiements avec stockage partagé, l'image (ou le volume) est accessible depuis tous les nœuds, ce qui simplifie le processus.
- 

## 2. Processus de migration pas à pas

### Étape 1 : Pré-migration

- **Vérification de la cohérence** : Les scripts ou outils de vérification précédemment mis en place confirment que la VM est dans un état stable (pas d'opérations critiques en cours, ressources réseau et stockage disponibles sur le nœud cible).
- **Validation des ressources** : Nova, via l'ordonnanceur, s'assure que le nœud cible dispose des ressources (CPU, mémoire) nécessaires.

### Étape 2 : Lancement de la migration par Nova

- **Commande et coordination** : Dès que la migration est lancée, Nova envoie une instruction au service nova-compute du nœud source pour préparer la migration.
- **Communication avec Xen** : Le driver XenAPI se charge d'interfacer avec l'hyperviseur Xen pour geler (dans le cas d'une migration à froid) ou synchroniser l'état de la VM (pour une live migration).

### Étape 3 : Transfert de l'état de la VM

- **Migration de la mémoire et de l'état CPU** :
  - *Live Migration* : La mémoire (et l'état CPU) est transférée de manière incrémentale vers le nœud cible. Une fois que la majorité de l'état est synchronisée, une phase finale de basculement est réalisée, minimisant ainsi le temps d'arrêt.

- *Migration à froid* : La VM est d'abord arrêtée, puis son état est copié (en particulier si le stockage n'est pas partagé).
- **Rôle de Xen** : La commande de migration dans Xen (souvent via des outils tels que `xl migrate` ou des équivalents gérés par XenAPI) prend en charge le transfert de l'état interne de la VM entre l'hôte source et l'hôte de destination.

#### Étape 4 : Transfert ou ré-association de l'image/disque

- **Scénario stockage partagé** :
  - Si la VM utilise un stockage partagé (par exemple, via Ceph, NFS ou un backend similaire), le disque (ou image) reste accessible depuis les deux nœuds. Ainsi, seule la mémoire et l'état de la VM sont migrés.
- **Scénario stockage local** :
  - Si l'image (ou le volume racine) est stockée en local sur l'hôte source, il est nécessaire de copier le support disque vers le nœud de destination. Cette opération peut être réalisée via une interaction entre nova-compute et Glance/Cinder, qui gère le transfert de l'image ou le réattachement du volume.
- **Impact sur l'image** :
  - Dans un contexte de migration avec stockage partagé, l'image initiale reste inchangée dans Glance et n'est pas modifiée.
  - En cas de copie, la cohérence de l'image doit être vérifiée afin de garantir que la nouvelle instance utilise bien la version à jour.

#### Étape 5 : Finalisation de la migration

- **Reprise sur le nœud cible** : Le service nova-compute sur l'hôte de destination finalise l'injection de l'état transféré dans une nouvelle instance.
  - **Mise à jour des services** :
    - **Nova Database** : Le statut de la VM est mis à jour (nouvelle localisation, état de migration achevé).
    - **Neutron** : Les règles et configurations réseau (adresses IP, routes) sont réappliquées pour s'assurer que la VM reste accessible.
    - **Cinder/Glance** : En fonction du mode d'accès au disque, ces services peuvent être sollicités pour garantir que le stockage est correctement monté et cohérent.
  - **Validation post-migration** : Des outils de monitoring et des scripts de vérification (similaires à ceux utilisés en phase de pré-migration) valident que la VM fonctionne correctement sur le nouvel hôte.
-

### 3. Points particuliers avec Xen

- **Driver spécifique :**
    - Contrairement à un environnement KVM/QEMU (souvent géré par libvirt), avec Xen, Nova utilise généralement un driver basé sur XenAPI. Celui-ci traduit les commandes de migration vers les appels spécifiques à Xen, comme la préparation d'un checkpoint et la transmission de l'état VM via le réseau.
  - **Limites techniques :**
    - La live migration avec Xen peut présenter des contraintes (par exemple, des exigences strictes sur la configuration du stockage partagé et de la mise en réseau). Certains environnements Xen ne supportent que la migration à froid si les conditions de live migration ne sont pas satisfaites.
  - **Configuration prérequis :**
    - Pour que la migration soit réussie, il faut s'assurer que les versions du logiciel de virtualisation, les outils Xen (XAPI, xl, etc.) et les configurations réseau soient harmonisées entre les hôtes source et destination.
- 

### Conclusion

En résumé, une migration de VM sur OpenStack avec Xen suit ces grandes étapes :

1. Validation préalable de l'état de l'instance et de la compatibilité des hôtes via des vérifications automatisées.
2. Initiation par Nova qui, en interaction avec son ordonnanceur et le service nova-compute (via le driver XenAPI), déclenche la migration.
3. Transfert de l'état (mémoire, CPU) et, si nécessaire, du support disque (dans le cas d'un stockage local) vers le nœud cible.
4. Finalisation de la migration avec la mise à jour des services OpenStack (Nova, Neutron, éventuellement Cinder/Glance) afin de garantir la continuité du service et l'intégrité de l'image d'origine.

Chaque composant OpenStack joue un rôle complémentaire pour s'assurer que la migration se fasse en douceur, même avec un hyperviseur Xen, en traduisant et adaptant les commandes de migration aux spécificités de cet environnement.

---

Cette description précise la chaîne complète d'événements et l'interaction entre Nova et les autres services d'OpenStack, en mettant l'accent sur les différences

liées à l'utilisation de Xen par rapport aux environnements plus classiques KVM/QEMU.

citeturn0search0